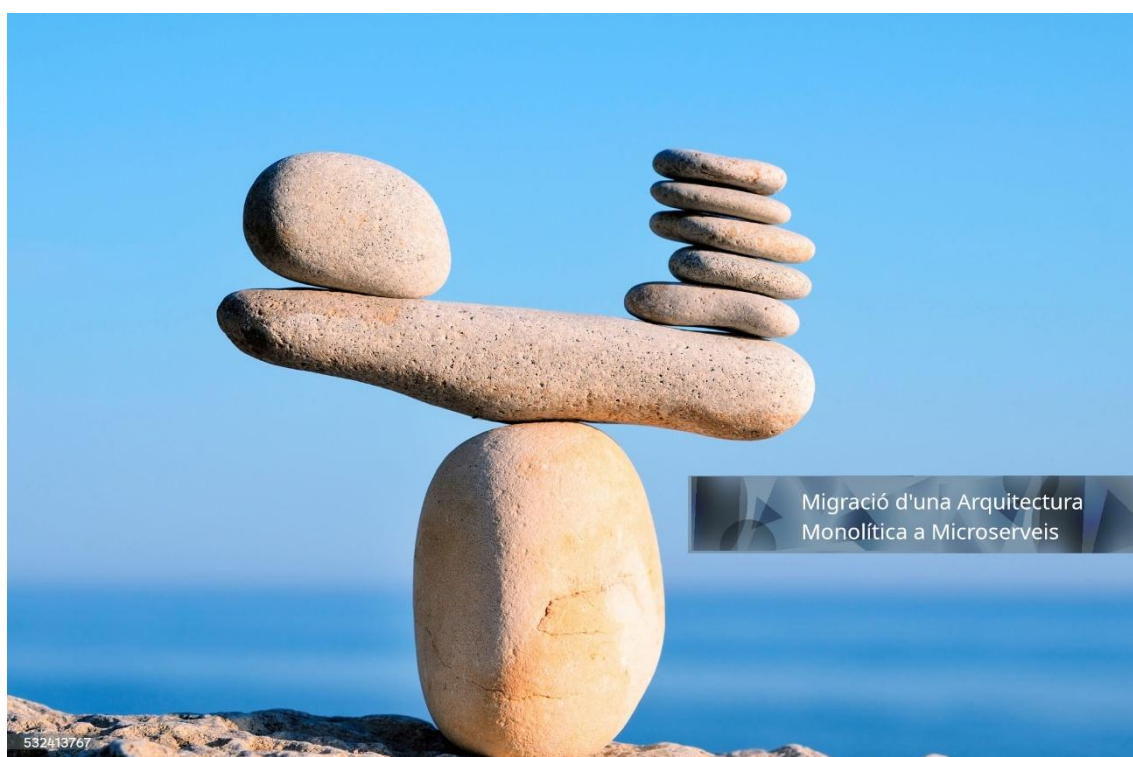




Migració d'una Arquitectura Monolítica a Microserveis

Sergio Santamaría Bayés

Grau en Enginyeria Informàtica

Àrea: GNU/Linux

Tutor: Joaquin Lopez Sanchez-Montañes

Data Lliurament: Gener de 2025

FITXA DEL TREBALL FINAL

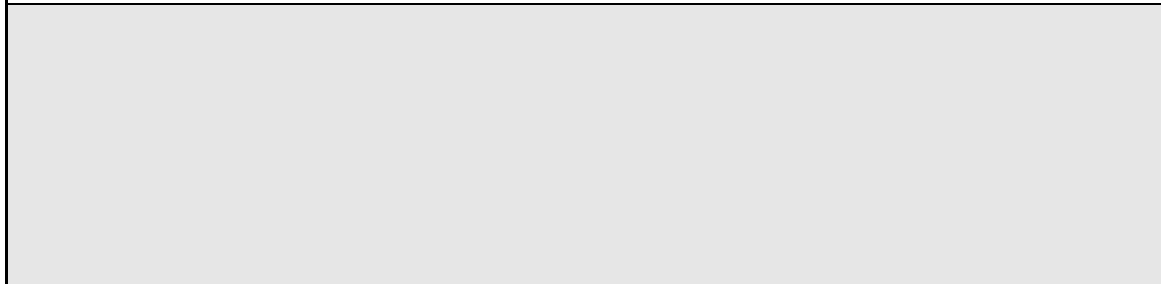
Títol del treball:	Migració d'una Arquitectura Monolítica a Microserveis
Nom de l'autor:	<i>Sergio Santamaría Bayés</i>
Nom del consultor/a:	Joaquin Lopez Sanchez-Montañes
Nom del PRA:	
Data de lliurament (mm/aaaa):	01/2025
Titulació:	Grau en Enginyeria Informàtica
Àrea del Treball Final:	<i>GNU/Linux</i>
Idioma del treball:	<i>Català</i>
Paraules clau	Microservices, Cloud, Arquitectura Monolítica, API, Kubernetes, Docker, HELM Charts, Pipelines CI/CD, DevOps, GitOps
Aquest treball explora la migració d'una aplicació monolítica clàssica cap a una arquitectura basada en microserveis, amb l'objectiu d'analitzar els beneficis i reptes d'aquesta transició. Aprofitant l'experiència adquirida en un projecte real, el context d'aplicació inclou una organització que busca modernitzar la seva infraestructura per millorar la gestió, la flexibilitat i l'eficiència energètica. La metodologia combina pràctiques modernes com <i>DevOps</i> i <i>GitOps</i> per facilitar la transició, amb una anàlisi detallada dels tres components principals de l'aplicació: <i>Front-End</i>, <i>Back-End</i> i base de dades. El projecte es divideix en fases, des del disseny i la planificació fins a la implementació i validació. S'inclouen eines com contenidors, plataformes de <i>Cloud Computing</i> i tècniques d'escalabilitat dinàmica. Els resultats esperats se centren en l'obtenció d'una arquitectura més robusta, flexible i econòmica, amb una optimització significativa dels recursos tecnològics i una reducció del consum energètic. També es preveu una millora en la capacitat de desplegament i manteniment de funcionalitats. Les conclusions apunten a com l'arquitectura de microserveis pot millorar la sostenibilitat tecnològica i ambiental, alhora que es redueixen costos operatius. Aquest treball no només documenta el procés de migració, sinó que també busca demostrar el valor d'aquesta transformació per a empreses que aspiren a mantenir-se competitives en un entorn tecnològic en constant evolució.	

This project explores the migration of a traditional monolithic application to a microservices-based architecture, focusing on analyzing the benefits and challenges of this transformation. Leveraging real-world experience from a current project, the application context involves an organization seeking to modernize its infrastructure to improve management, flexibility, and energy efficiency.

The methodology incorporates modern practices such as DevOps and GitOps to streamline the transition, along with a detailed analysis of the three main application components: Front-End, Back-End, and database. The project is structured into phases, from design and planning to implementation and validation, employing tools like containers, cloud computing platforms, and dynamic scalability techniques.

Expected results include a more robust, flexible, and cost-effective architecture, significantly optimized technological resources, and reduced energy consumption. Additionally, enhanced deployment and maintenance capabilities for application functionalities are anticipated.

The conclusions underline how microservices architecture can enhance technological and environmental sustainability while reducing operational costs. This project not only documents the migration process but also aims to demonstrate the value of such a transformation for businesses striving to remain competitive in an ever-evolving technological landscape.



Índex

1. Introducció.....	6
1.1 Context i justificació del Treball.....	6
1.2 Objectius del Treball.....	6
1.3 Enfocament i mètode seguit	7
1.4 Planificació del Treball.....	14
1.5 Breu sumari de productes obtinguts.....	16
1.6 Breu descripció dels altres capítols de la memòria	17
2. Descripció de les Eines i Arquitectura	18
2.1 Arquitectura de Microserveis	18
2.2 Gestió d'APIs i Comunicacions Internes.....	19
2.3 Bases de Dades Independents i descentralitzades.....	21
2.4 Contenedors i Orquestració de Microserveis	23
2.5 Integració i Desplegament Continus	27
3. Disseny del Sistema	30
3.1 Identificació de Components	30
3.2 Estratègia de Seguretat i Autenticació	43
3.3 Estratègia de Monitorització i Registre.....	45
4. Planificació Temporal i Recursos	45
4.1 Desglossament de Fases del Projecte	45
4.2 Estratègia d'Integració	52
4.3 Pla de Proves i Integració	58
5. Avaluació i Gestió de Riscos	68
5.1 Identificació de Biaixos i Mesures Pal·liatives.....	68
5.2 Compliment Normatiu i Auditoria.....	72
6. Futur del Projecte.....	80
6.1 Resum del Progrés.....	81
6.2 Projecció de Millores Potencials i Ampliacions	81
7. Conclusió.....	83
8. Glossari.....	85
9. Bibliografia	90

1. Introducció

1.1 Context i justificació del Treball

En un entorn de creixement tecnològic constant, les empreses es veuen en la necessitat de gestionar una quantitat creixent de dades sensibles i confidencials, com ara dades d'usuaris i enregistraments de vídeo, en entorns cada cop més complexos. La normativa legal sobre protecció de dades, especialment en àmbits com el Registre General de Protecció de Dades (RGPD) de la Unió Europea, exigeix un alt nivell de control i seguretat per garantir el respecte als drets de privacitat dels individus. A més, la proliferació de tecnologies de monitorització (càmeres, sensors, etc.) i l'emmagatzematge d'informació en entorns digitals intensifiquen la necessitat de tenir solucions segures i eficients.

Aquest projecte neix per donar resposta a aquestes necessitats, facilitant una estructura tecnològica que permeti la integració i gestió de sistemes de dades a través d'una arquitectura de microserveis. Aquesta aproximació es fa necessària per garantir que cada component sigui escalable, segur i adaptable als canvis normatius i tecnològics que es puguin presentar.

L'objectiu d'aquest projecte és, per tant, desenvolupar una solució integral que permeti una gestió de dades flexible, orientada a minimitzar els riscos associats al tractament de dades personals i acomplir amb les normatives vigents de protecció de dades. A més, es busca que aquesta estructura pugui adaptar-se a futures necessitats, donant a les empreses un avantatge competitiu en un mercat cada cop més exigent en termes de seguretat i privacitat de la informació.

1.2 Objectius del Treball

Els objectius del projecte es poden dividir en tres blocs principals:

Objectius tècnics:

Analitzar les diferències entre l'arquitectura monolítica i la de microserveis, destacant els beneficis i reptes de la migració. Implementar una migració de prova d'una aplicació fictícia però amb base real, utilitzant eines i tecnologies de plataforma amb arquitectura de microserveis.

Objectius econòmics i mediambientals:

Avaluar l'impacte en costos operatius i energètics abans i després de la migració, destacant l'optimització de recursos i la reducció de l'impacte ambiental. Fer ús de la metodologia Green IT, per assegurar que la migració contribueixi a un ús més eficient dels recursos tecnològics reduint l'empremta de carboni.

Objectius humans:

Anàlisi de la gestió del canvi de la organització i del personal que administra i gestiona el projecte IT. S'haurà d'avaluar com serà la comunicació que serveixi per mitigar les expectatives i incerteses de l'organització en relació amb el canvi, doncs és crítica per eliminar el soroll que es genera en el procés.

1.3 Enfocament i mètode seguit

Per establir una transició efectiva d'una arquitectura monolítica a una basada en microserveis, cal definir les implicacions tècniques clau de cada component i servei que participaran en el projecte. El mètode seguit realitza una anàlisi exhaustiva d'aquestes implicacions per assegurar que tots els requisits del sistema s'ajustin a les necessitats de l'organització i als processos de transformació.

Descomposició de la Funcionalitat Actual en Microserveis

Identificació de Mòduls i Components Existents: L'anàlisi de les dependències dins de l'arquitectura monolítica permet descompondre la funcionalitat en mòduls autònoms. Aquest procés implica identificar aquells components que poden funcionar independentment, que seran la base dels futurs microserveis.

Requeriments d'Escalabilitat i Interoperabilitat: Cada microservei ha de ser escalable i interoperable amb altres serveis. És necessari establir especificacions per a l'escalabilitat independent de cada servei, així com els requeriments d'interacció a través d'APIs (Interfícies de Programació d'Aplicacions).

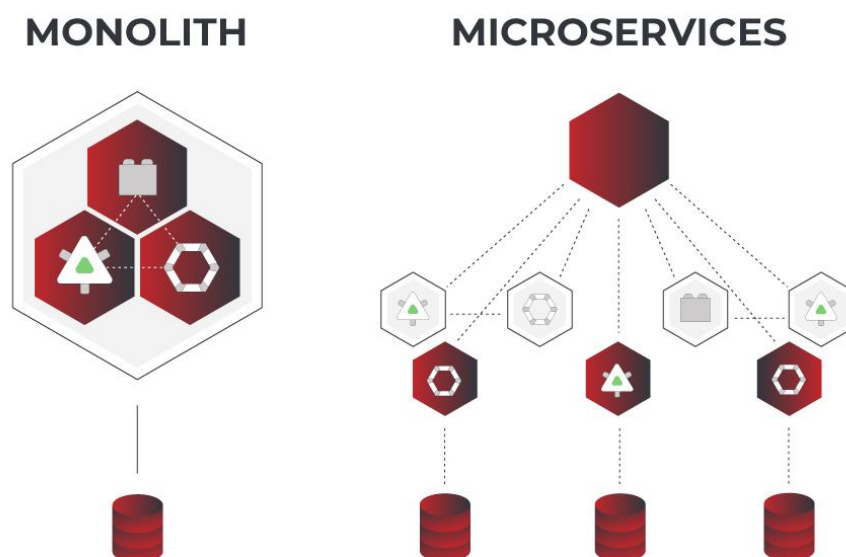


Figura 1: [Monolithic vs Microservices architecture](#)

Disseny i Implementació d'APIs

API Gateway i Gestió de Trafic: L'API Gateway actuarà com a únic punt d'entrada, encarregant-se de la gestió de peticions i el control d'accés. Ha de ser capaç de gestionar la latència, la seguretat, i l'equilibrat de càrrega entre els serveis.

Protocol de Comunicació: Els microserveis utilitzen protocols com REST o gRPC per a la comunicació, però és essencial definir quin protocol s'adapta millor a cada tipus d'interacció, valorant l'eficiència, la seguretat i els requeriments de rendiment.

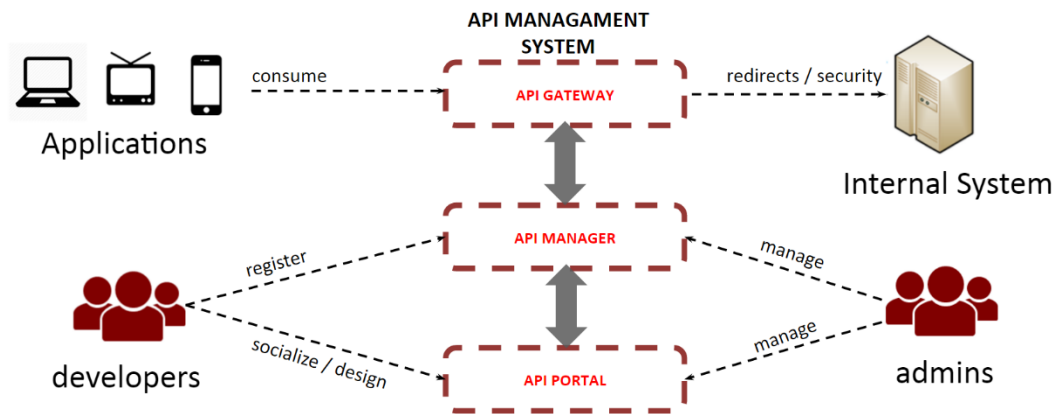


Figura 2: [API Management: ¿qué es y para qué sirve?](#)

Arquitectura de Bases de Dades

Descentralització de Dades: Mentre que les arquitectures monolítiques solen centralitzar la base de dades, en microserveis es recomana utilitzar bases de dades independents per a cada servei, mantenint la integritat i consistència de les dades.

Sincronització de Dades i Consistència: Cal implementar patrons de consistència eventual, sincronització de dades i control de transaccions entre bases de dades. Això implica definir estratègies d'integració de dades que garanteixin consistència i fiabilitat.

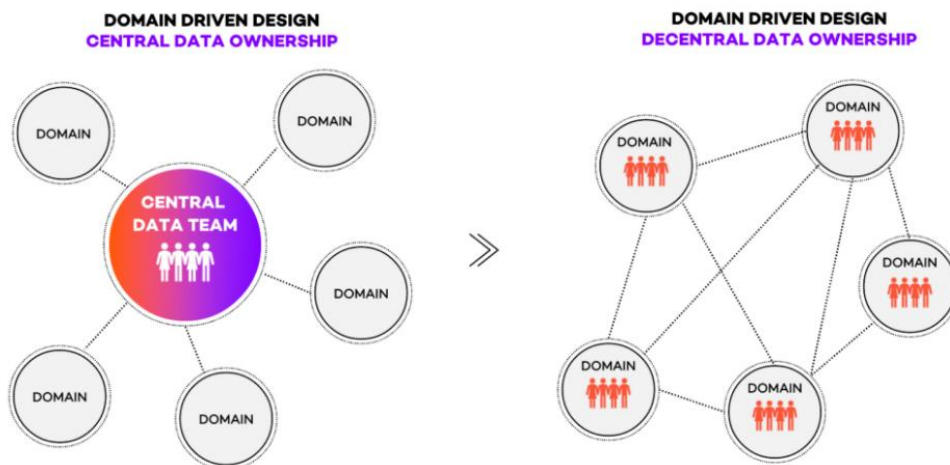


Figura 3: [From centralized data to decentralized ownership](#)

Gestió d'Autenticació i Autorització

Control d'Accés Segur: En una arquitectura de microserveis, és fonamental gestionar l'autenticació i l'autorització a través de protocols com OAuth 2.0 o OpenID Connect, per assegurar un accés segur entre els diferents serveis.

Gestió de Secrets i Credencials: Cada microservei necessita accés segur als secrets i credencials, i aquests han de ser gestionats a través de serveis de vaulting (com HashiCorp Vault o AWS Secrets Manager) que ofereixen accés segur i controlat.

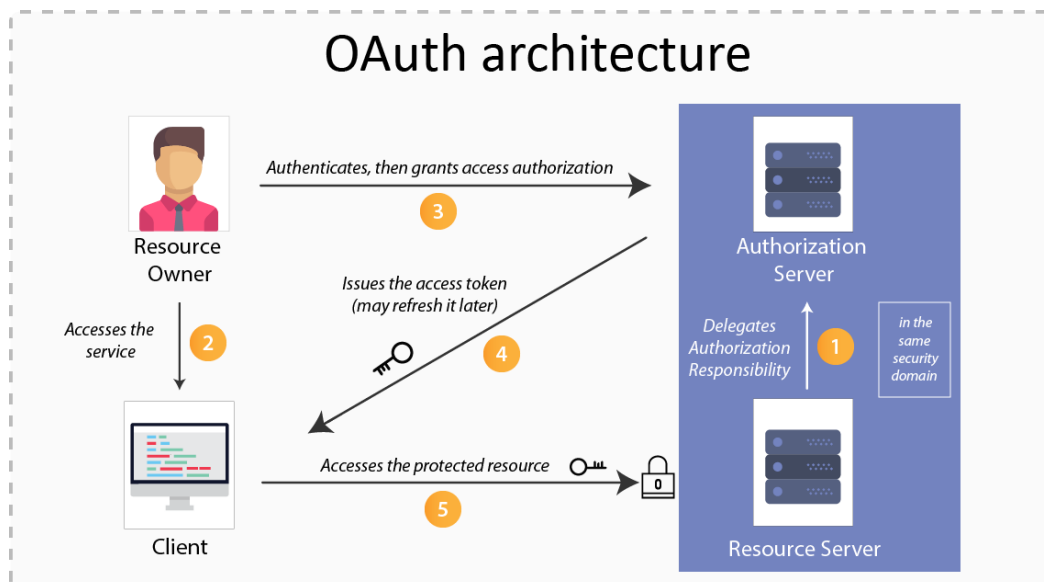


Figura 4: [Spring Security with OAuth2](#)

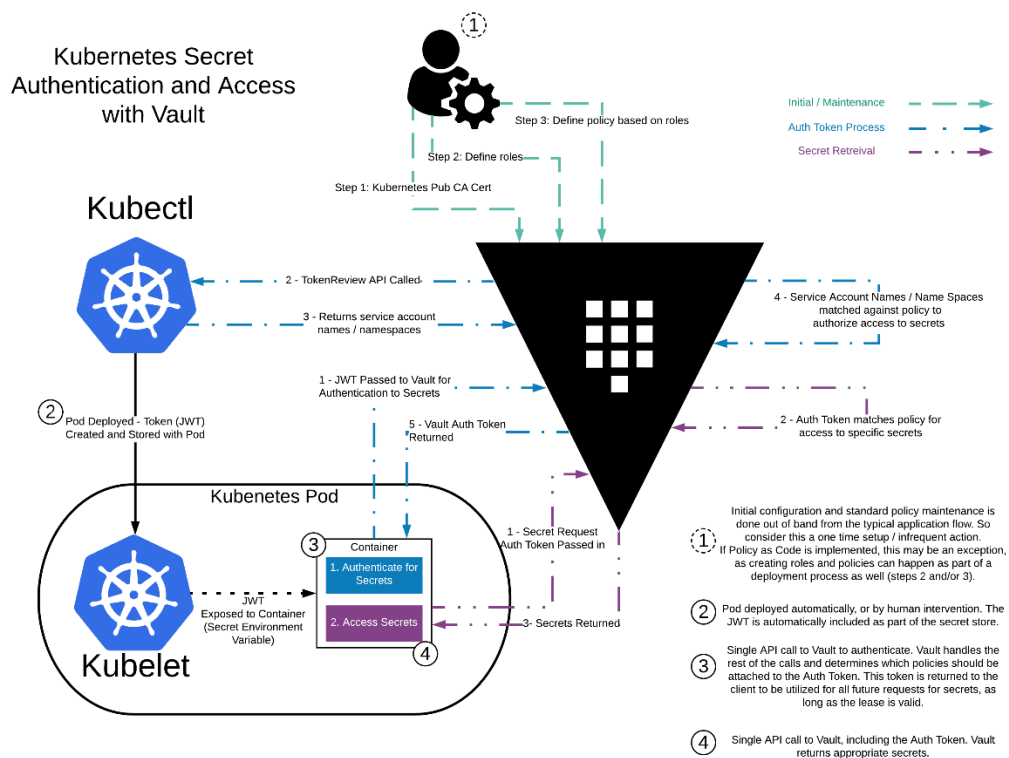


Figura 5: [Kubernetes integration with external Vault - devopstales](#)

Gestió de Contenidors i Orquestració

Contenidors Docker i Orquestració amb Kubernetes: Els microserveis s'implementen en contenidors Docker, permetent un desplegament aïllat i escalable. Kubernetes s'utilitzarà per a la gestió de l'orquestració, el desplegament i l'escalabilitat.

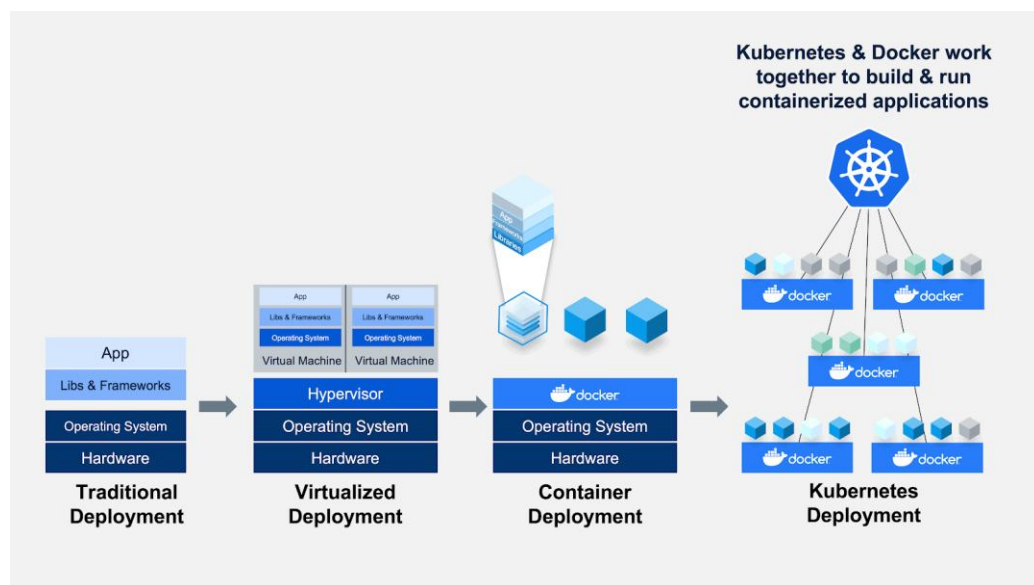


Figura 6: [Docker and Kubernetes | How are they better together?](#)

Estratègia de Monitorització i Registre (Logging): És crucial implementar una estratègia robusta de monitorització i registre. Solucions com Prometheus per a la monitorització i ELK Stack (Elasticsearch, Logstash, Kibana) per al registre proporcionen visibilitat sobre l'operativa dels serveis.

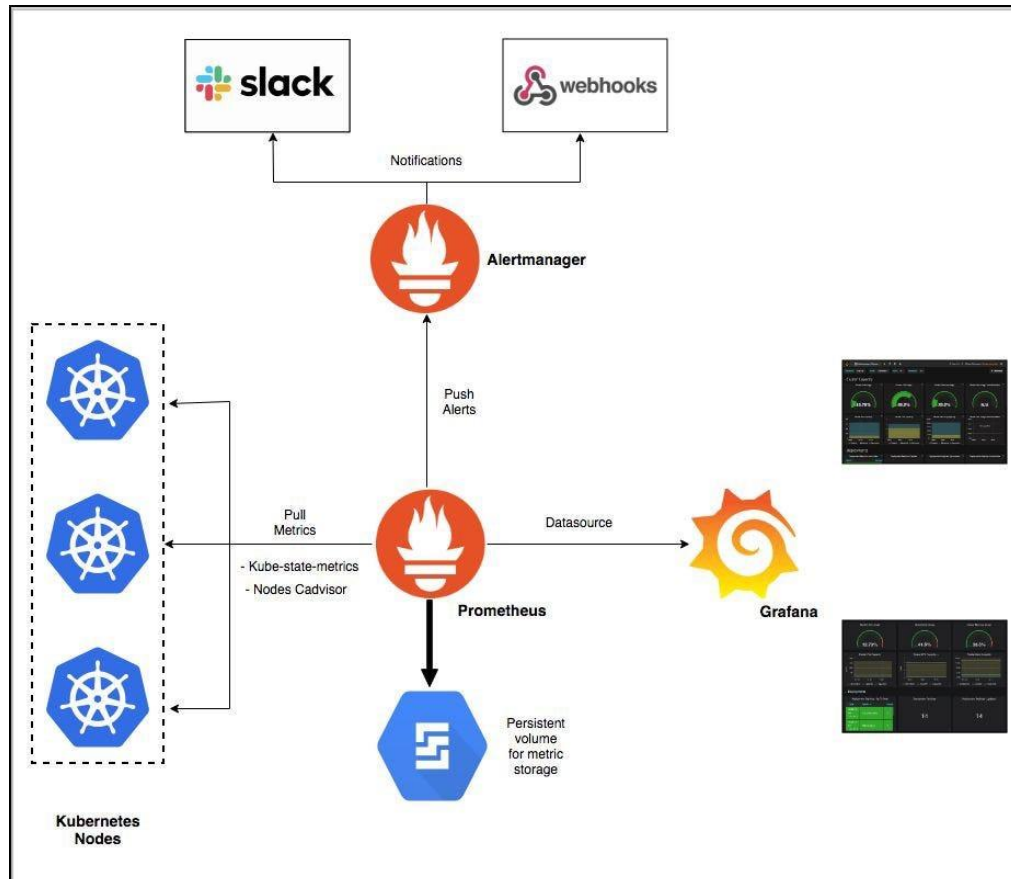


Figura 7: [How to Monitor Kubernetes Cluster with Prometheus and Grafana?](#)

Infraestructura per Proves i CI/CD

Integració i Desplegament Continus (CI/CD): Els processos de CI/CD permeten un desplegament ràpid i segur. Jenkins, GitLab CI/CD o AWS CodePipeline són solucions que integren desenvolupament, proves i desplegament en un únic flux automatitzat.

Estratègia de Proves de Contracte: Cada microservei ha de ser validat de forma independent per evitar efectes col·laterals durant la integració amb altres serveis, assegurant la compatibilitat i el correcte funcionament entre interfícies.

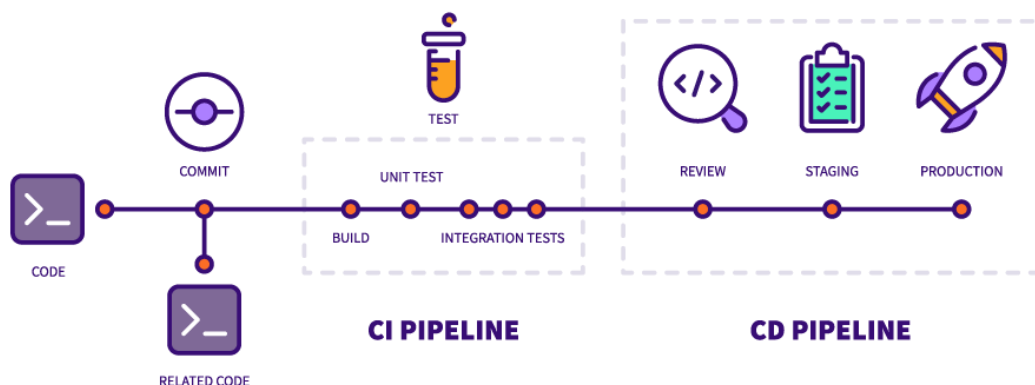


Figura 8: [GitLab CI and CD Pipeline](#)

Normativa de Compliment i Seguretat

Compliment de Normativa de Dades: El projecte ha de complir amb normatives de protecció de dades, com el GDPR, per a la gestió segura de dades personals i d'altres dades sensibles.

Revisions i Auditories Regulars: Implementar auditories i revisions periòdiques dels processos per assegurar el compliment de les normatives. Això inclou la seguretat en el processament de dades i l'ús de serveis amb registre encriptat.



Figura 9 : [Assurance & Compliance | interact.training](#)

Requeriments de Hardware i Infraestructura Física

Definició dels Requeriments d'Infraestructura IT: Cal identificar els requisits d'infraestructura necessaris per suportar els serveis de manera òptima, ja sigui on-premises o al núvol. Aspectes com la capacitat de processament, memòria, i disponibilitat de xarxa són essencials.

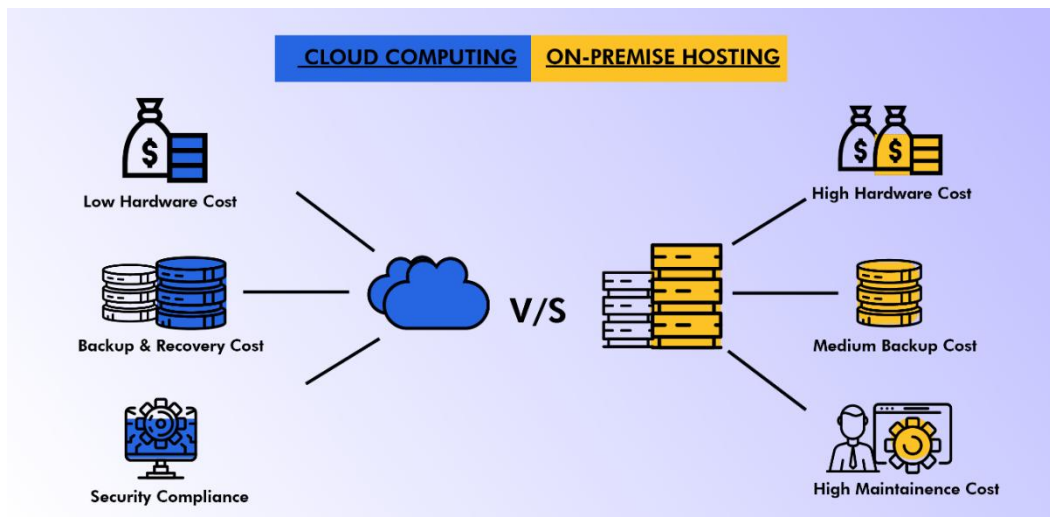


Figura 10: On-Premise vs Cloud-Hosted- Which Application Suits you the Most?

1.4 Planificació del Treball

La planificació temporal per a aquest projecte es defineix en funció de les diferents fases del projecte, assegurant que totes les activitats es puguin dur a terme de manera òptima i que es compleixin els terminis establerts. A continuació es detalla la cronologia de les diferents fases, així com els objectius principals de cada una d'elles. Els terminis s'han plantejat per complir amb els requisits tècnics i de qualitat, i s'han considerat recursos addicionals per gestionar possibles desviacions.

[illegible]

[illegible]

1.5 Breu sumari de productes obtinguts

PAC1: Pla de Treball

En la primera fase, corresponent a la PAC1: Pla de Treball, es va crear un document que defineix clarament els objectius generals del projecte, el seu abast i la metodologia que es seguirà. A més, es va establir un cronograma que organitza de manera estructurada les activitats a realitzar. Aquestes fites inicials són essencials per garantir que totes les fases posteriors es desenvolupin de manera alineada amb els objectius establerts. També es va esbossar una proposta inicial per a la infraestructura tècnica requerida, marcant el punt de partida per al disseny tècnic.

PAC2: Disseny i Implementació

Durant la PAC2: Disseny i Implementació, es van assolir avenços significatius en la configuració de l'arquitectura basada en microserveis. Es va elaborar un disseny detallat d'aquesta arquitectura, posant especial èmfasi en la modularitat i l'escalabilitat. Així mateix, es van implementar els primers prototips de microserveis en un entorn de prova, utilitzant eines com GitLab Pipelines i ArgoCD per automatitzar els processos de desenvolupament i desplegament. Es van dur a terme validacions inicials per assegurar el compliment de normatives de seguretat i protecció de dades.

PAC3: Resultats i Anàlisi

En la fase corresponent a la PAC3: Resultats i Anàlisi, es van realitzar proves exhaustives per avaluar el rendiment i l'escalabilitat dels microserveis, comparant-los amb les limitacions de l'arquitectura monolítica. Aquestes proves van permetre elaborar un informe detallat que no només identifica els beneficis clars de la nova arquitectura, sinó que també documenta les incidències trobades i les solucions implementades per abordar-les.

PAC4: Lliurament Final de la Memòria

Finalment, a la PAC4: Lliurament Final de la Memòria, es va confeccionar una memòria completa que inclou els resultats obtinguts, les conclusions del projecte i recomanacions per al seu manteniment i possible evolució. Aquest document va ser complementat amb una guia tècnica per a la gestió i ús de la nova arquitectura, així com amb una presentació visual per a la defensa final del treball.

1.6 Breu descripció dels altres capítols de la memòria

Introducció

Aquest capítol ofereix una visió general del projecte, incloent-hi el context, els objectius i les motivacions personals per abordar la migració d'una arquitectura monolítica a una basada en microserveis. També introdueix el marc conceptual del treball i defineix l'abast, delimitant clarament les àrees en què se centra l'estudi. Serveix com a base per a entendre la importància del projecte i la seva rellevància tant tècnica com pràctica.

Estat de l'Art

Aquí es presenta una anàlisi de les arquitectures monolítiques i de microserveis, explorant els seus avantatges i desavantatges. També es revisen les tecnologies i eines clau utilitzades en el projecte, com ara contenidors, orquestració amb Kubernetes, GitLab Pipelines, HELM Charts i ArgoCD. Aquest capítol contextualitza el treball dins les tendències actuals de desenvolupament de programari i justifica la migració plantejada.

Metodologia

Es detalla l'enfocament seguit per a la migració, incloent-hi les etapes de disseny, desenvolupament i validació. S'expliquen les tècniques d'integració contínua (CI) i desplegament continu (CD) implementades, així com les estratègies per garantir el compliment de normatives com el RGPD. Aquest capítol connecta el plantejament teòric amb la seva aplicació pràctica.

Disseny de l'Arquitectura

Aquest apartat aborda el disseny detallat de l'arquitectura basada en microserveis, incloent-hi la divisió del sistema en mòduls, les bases de dades independents, i la gestió de comunicacions internes amb APIs. També es descriu l'estructura tècnica utilitzada per assegurar escalabilitat i robustesa. Té un paper central, ja que és el punt de partida per a la implementació.

Implementació i Migració

Es descriuen els passos seguits per migrar el sistema des de l'arquitectura monolítica, incloent-hi la creació de microserveis, la configuració dels contenidors i l'orquestració. També es documenten els problemes trobats durant aquesta fase i les solucions adoptades. Aquest capítol és crucial perquè demostra com la teoria es tradueix en accions concretes.

Validació i Resultats

Aquí es mostren les proves realitzades per validar l'arquitectura migrada, amb una comparativa del rendiment i l'eficiència entre l'antiga arquitectura monolítica i la nova

basada en microserveis. També s'analitzen les incidències resoltes i es presenten els resultats clau que confirmen els beneficis de la migració.

Conclusions i Futurs Desenvolupaments

El capítol final resumeix els resultats obtinguts, destacant els beneficis de l'arquitectura de microserveis en termes de flexibilitat, escalabilitat i estalvi energètic. A més, planteja possibles millores i ampliacions futures per optimitzar encara més el sistema i les seves aplicacions. Aquest capítol reflexiona sobre l'impacte global del treball i ofereix una visió a llarg termini.

2. Descripció de les Eines i Arquitectura

2.1 Arquitectura de Microserveis

La implementació del projecte es realitzarà amb una arquitectura de microserveis. Això permet que els diferents components siguin desenvolupats, desplegats i mantinguts de manera independent, millorant l'escalabilitat i resiliència del sistema. Els microserveis estan estructurats per funcions específiques, com la gestió de dades d'usuaris, la configuració de registres de vídeo i la integració amb sistemes de bases de dades externes.

Una arquitectura de microserveis és un enfocament dissenyat per dividir una aplicació monolítica en una sèrie de serveis petits, independents i autònoms. A diferència d'un sistema monolític, on tota la funcionalitat de l'aplicació es troba en un únic sistema o codi font, en una arquitectura de microserveis, cada component té una responsabilitat ben definida i es comunica amb els altres a través de APIs. Això permet que els serveis siguin més flexibles i fàcilment escalables. A més, cada microservei pot ser desenvolupat, desplegat i escalat de manera independent, la qual cosa facilita la gestió i el manteniment de l'aplicació en general.[\[1\]](#)

Una de les avantatges més destacades d'adoptar una arquitectura de microserveis és la capacitat d'escalar cada servei de manera independent, en funció de la seva demanda. Si un microservei experimenta un augment de trànsit o càrrega, només cal escalar aquell servei en particular, sense necessitat de tocar tota l'aplicació. Això és especialment important en el nostre projecte de migració, ja que ens permet optimitzar l'ús de recursos de manera més eficient.

En una arquitectura monolítica, una fallada en un component pot afectar tota l'aplicació. En canvi, els microserveis operen de manera aïllada, de manera que la fallada d'un servei no afectaria els altres, millorant la resiliència global del sistema. Això és fonamental en un projecte de migració, ja que permet una transició més segura i estable entre els diferents components de l'aplicació.

Cada microservei pot ser actualitzat i desplegat de manera independent, sense afectar altres parts del sistema. Això fa que els desplegaments siguin més ràpids i menys arriscats, ja que només cal provar i desplegar una petita part de l'aplicació en lloc de tot el conjunt.

Una altra característica important de l'arquitectura de microserveis és la llibertat de triar la tecnologia que millor s'adapti a cada servei. Com cada microservei és independent, podem utilitzar diferents llenguatges de programació, frameworks i bases de dades per a cada un, en funció de les necessitats específiques del servei.

Les arquitectures de microserveis poden resultar més eficients en termes de costos operatius i energètics. En un entorn de microserveis, els serveis poden ser gestionats i desplegats en entorns de núvol amb escalabilitat dinàmica, aprofitant millor els recursos i reduint el malbaratament de capacitat.

En el nostre projecte de migració d'una arquitectura monolítica a microserveis, l'objectiu principal és aprofitar tots aquests beneficis. Ens centrem en la creació d'un conjunt de serveis independents que poden ser desenvolupats, provats, i desplegats de manera autònoma. Això ens permetrà no només millorar la flexibilitat i la mantenibilitat, sinó també augmentar l'escalabilitat i la resiliència del sistema a mesura que evolucionem.[\[2\]](#)

A més, en aquest procés, utilitzarem eines modernes de gestió de microserveis, com Docker per a la creació de contenidors i Kubernetes per a l'orquestració i gestió d'aquests contenidors. Això ens permetrà una gestió més eficient dels microserveis i ens ajudarà a implementar solucions com la CI/CD, facilitant el desplegament continu i la integració contínua a mesura que implementem els nous microserveis.

2.2 Gestió d'APIs i Comunicacions Internes

En el procés de migració d'una arquitectura monolítica cap a una arquitectura basada en microserveis, la gestió d'APIs i les comunicacions internes és un dels aspectes més crítics per garantir un sistema robust i eficient. A diferència del model monolític, on tots els components conviuen i interactuen en un únic espai de memòria, els microserveis són components independents que necessiten maneres estructurades de comunicar-se entre si. Això ens permet una major flexibilitat i modularitat, però també introdueix complexitats que cal gestionar adequadament.

Les APIs són el mitjà principal pel qual els microserveis es comuniquen. En aquest context, cada microservei exposa una API que defineix les operacions que ofereix, com ara recuperar dades, enviar informació o interactuar amb altres serveis. Aquestes APIs poden seguir el model RESTful, que utilitza HTTP i formats de dades com JSON, o utilitzar alternatives més eficients com gRPC, que és especialment útil en sistemes amb alta intensitat de comunicacions.

Documentar i gestionar aquestes APIs és fonamental. En el nostre projecte, utilitzem eines com Swagger o OpenAPI per assegurar que cada servei té una descripció clara i accessible. Aquesta documentació és especialment útil per als desenvolupadors, que poden consultar com integrar-se amb altres serveis sense

ambigüitats. També hem de tenir en compte la seguretat, implementant mecanismes com tokens d'autenticació (JWT) per protegir la informació en trànsit. Els JWT es divideixen en tres parts: capçalera (header), contingut (payload amb claims) i firma (signature). S'utilitzen principalment per a l'autenticació i autorització. [3]

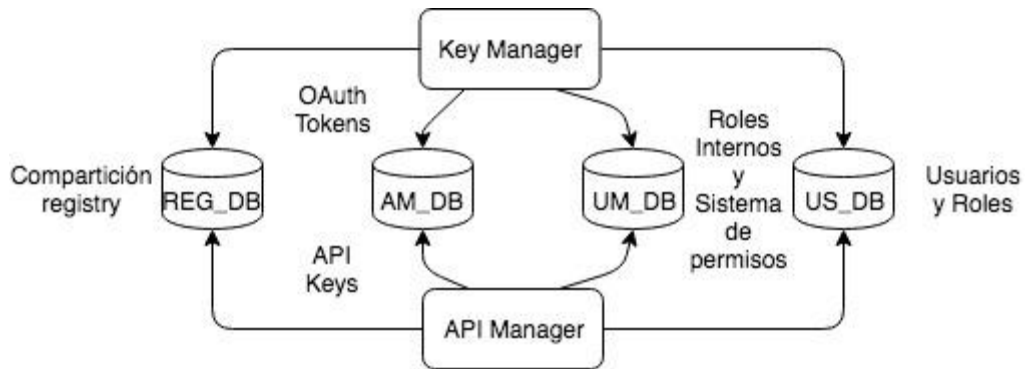


Figura 11 – Intercambio de datos entre API Manager e Identity Server

A més de les APIs, els serveis necessiten establir com es comuniquen entre si. Aquesta comunicació pot ser síncrona o asíncrona. Les comunicacions síncrones, com les basades en REST, són més directes però poden generar colls d'ampolla si un servei no està disponible. Per altra banda, les comunicacions asíncrones, que utilitzen sistemes de missatgeria com Kafka o RabbitMQ, permeten que els serveis enviïn missatges sense esperar una resposta immediata, cosa que augmenta la resiliència del sistema. Al següent [apartat 2.3](#) explicarem en més profunditat aquestes eines de gestió de dades descentralitzades.

Un exemple del nostre projecte és l'ús de patrons com Publish/Subscribe (més conegut com només **pub/sub**), on un servei envia notificacions (per exemple, sobre canvis en una base de dades) que altres serveis poden rebre segons els seus interessos. Això ens permet desacoblar els serveis i assegurar que la informació arriba només a qui la necessita.

Per gestionar aquestes comunicacions de manera més eficient, hem implementat un API Gateway que centralitza i controla tot el tràfic entre serveis. Aquesta eina simplifica l'accés als serveis des de l'exterior, afegint capes de seguretat, control d'accés i registre de peticions. També considerem l'ús d'un Service Mesh, com Istio, que gestiona automàticament el trànsit entre microserveis i afegeix funcionalitats com l'encryptació i el balanç de càrrega.

Aquest enfocament ens permet transformar el nostre antic sistema monolític en una estructura flexible i escalable. La modularitat dels microserveis facilita afegir o modificar funcionalitats sense afectar altres parts del sistema, i les comunicacions ben dissenyades garanteixen que el conjunt funcioni de manera fluida. Això també millora l'escalabilitat i la fiabilitat, factors crítics per al creixement futur de l'aplicació.

2.3 Bases de Dades Independents i descentralitzades

En la transició des d'una arquitectura monolítica a una basada en microserveis, un dels reptes més significatius és la gestió de les bases de dades. Mentre que un monòlit utilitza típicament una base de dades centralitzada, els microserveis es beneficien d'un enfocament descentralitzat, on cada servei gestiona la seva pròpia base de dades. Aquesta estratègia ofereix una major flexibilitat i escalabilitat, però també requereix noves metodologies de gestió de dades.

En el procés de migració del monòlit a microserveis, passem d'una base de dades única que gestionava totes les funcions del sistema a un model descentralitzat. Això ens permet dividir dades per dominis on cada domini de negoci (capa de presentació, lògica i dades) té el seu propi espai de dades. Alhora es poden estalviar recursos en optimitzar el consum d'infraestructura escalant només les bases de dades necessàries. Per últim, també facilita el manteniment: si una base de dades necessita una actualització o canvi de tecnologia, aquest procés no afecta la resta del sistema.

Aquesta estratègia no només millora la flexibilitat i la resiliència de l'aplicació, sinó que també ens prepara per a futures ampliacions, ja que cada microservei i la seva base de dades poden evolucionar de manera independent.

Cada microservei en l'arquitectura migrada és responsable de la seva pròpia funcionalitat i, per extensió, de les seves dades. Això implica que:

- Cada microservei té la seva pròpia base de dades: Això pot significar un tipus diferent de base de dades en funció de les necessitats del servei (per exemple, SQL per a transaccions financeres i NoSQL per a dades no estructurades).
- Desacoblament complet: Els microserveis no comparteixen esquemes ni accedeixen directament a les dades d'altres serveis. Aquest enfocament millora l'autonomia i minimitza l'impacte de canvis en un servei sobre els altres.

Com a principals avantatges obtenim una escalabilitat específica, on cada base de dades es pot escalar independentment segons les necessitats del microservei. Esdevé doncs, una funcionalitat més optimitzada atès que les bases de dades poden ser seleccionades en funció de les necessitats concretes del servei, millorant el rendiment general. Finalment, obtenim una reducció de dependències perquè els equips poden treballar de manera més independent, accelerant el desenvolupament i el desplegament.

Les dades descentralitzades introdueixen nous reptes i avantatges: en lloc de tenir un únic repositori de dades centralitzat com en un monòlit, les dades es distribueixen entre les bases de dades dels diferents microserveis. Això comporta diferents beneficis com ara resiliència i adaptabilitat. En cas de fallada d'un microservei o de la seva base de dades, els altres serveis poden continuar

funcionant. A més, l'arquitectura basada en dades descentralitzades permet canvis en un servei sense impactar altres serveis.

En un entorn distribuït, garantir que totes les còpies de les dades estiguin sincronitzades pot ser complex. Això requereix models de consistència eventual o estratègies específiques per manejar la latència. Quant a la gestió de transaccions, tenint en compte que s'estenen per múltiples serveis, han de ser gestionades amb patrons com la saga pattern, que divideix les transaccions en una sèrie d'accions independents però coordinades. Saga pattern permet mantenir la consistència eventual en sistemes distribuïts mitjançant l'execució de transaccions i, en cas d'errors, aplicant accions compensatòries per revertir operacions parcials. L'elecció entre orquestració o coreografia depèn de factors com la complexitat del sistema, els requeriments de desacoblament i les necessitats de control. [\[4\]](#)

Utilitzar eines com Apache Kafka o RabbitMQ permet coordinar canvis entre bases de dades de microserveis sense necessitat de dependències directes. Kafka és una plataforma de streaming distribuïda que permet publicar i subscriure's a registres, emmagatzemar-los de manera fiable i processar fluxos de dades en temps real. En una arquitectura de microserveis, Kafka facilita la comunicació asíncrona, desacoblant els serveis per garantir l'escalabilitat, la resiliència i la tolerància a errors. A través de patrons com la publicació i la subscripció a esdeveniments, els serveis poden intercanviar dades de manera eficient mitjançant temes Kafka.

Amb Spring Boot, Kafka proporciona una integració àgil que inclou configuracions senzilles per a consumidors i productors. Per exemple, es poden definir fàbriques de consumidors i productors utilitzant anotacions com `@EnableKafka` i configurar els respectius serveis per publicar o escoltar missatges de temes específics. Aquesta configuració, combinada amb Spring Kafka, permet implementar sistemes de missatgeria robustos en microserveis, capaços de gestionar alts volums de dades amb baixes latències. [\[5\]](#)

D'altra banda, les API inter-microserveis continuen sent útils per exposar dades específiques entre serveis i consumir només la informació necessària. Per reduir encara més la latència, és habitual utilitzar solucions de replicació o cache distribuïda com Redis, que complementen l'ús de Kafka per accelerar l'accés a les dades.

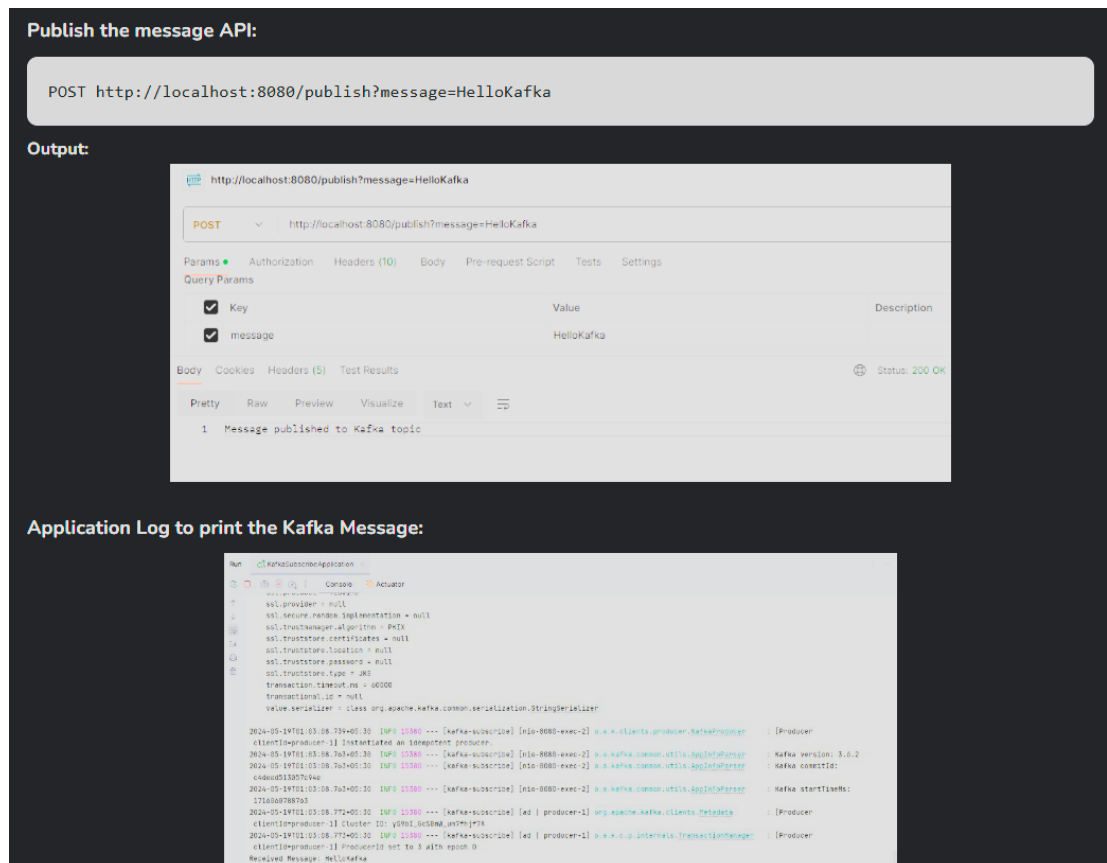


Figura 12 – Aplicació Java que envia missatges a Kafka

2.4 Contenedors i Orquestració de Microserveis

En la nostra migració d'una arquitectura monolítica a una basada en microserveis, els contenidors i les plataformes d'orquestració juguen un paper fonamental per aconseguir la modularitat, escalabilitat i gestió eficient dels serveis independents.

Els contenidors, com els proporcionats per Docker, encapsulen cada microservei juntament amb les seves dependències, garantint que es comportin de manera consistent independentment de l'entorn en què s'executin.

Avantatges dels contenidors en aquest context:

- **Aïllament total:** Cada microservei funciona en el seu propi contenidor, reduint conflictes entre dependències i permetent iteracions independents.
- **Portabilitat:** Els contenidors asseguren que els microserveis puguin ser executats en qualsevol infraestructura compatible, ja sigui un entorn local o al núvol.
- **Velocitat de desplegament:** En encapsular els serveis en imatges lleugeres, el temps de desplegament es redueix significativament comparat amb els entorns monolítics.

En la migració, **els microserveis es poden dividir progressivament del monòlit**, empaquetant-los com a contenidors i desplegant-los en entorns de proves o productius per separat.

Quan el nombre de microserveis creix, la gestió manual dels contenidors esdevé complexa. Aquí és on l'orquestració amb Kubernetes és essencial. Kubernetes permet ajustar automàticament el nombre de rèpliques de cada microservei en funció de la càrrega, assegurant eficiència i rendiment.

Kubernetes és un orquestrador de contenidors que s'ha convertit en l'estàndard de facto per al desplegament d'aplicacions al núvol. És un projecte de codi obert desenvolupat inicialment per Google, com una evolució de Borg, l'orquestrador utilitzat per Google en producció, però adaptat per treballar amb contenidors Docker.[6] Per tant, Kubernetes neix amb anys d'experiència de Google en l'execució de contenidors en entorns productius, cosa que el fa molt potent i altament escalable.

La funcionalitat principal de Kubernetes és el llançament i la gestió de contenidors en un clúster, permetent l'execució de múltiples contenidors a cada node de manera declarativa. Això significa que es defineix el nombre d'instàncies d'un contenidor que es vol executar, i Kubernetes s'encarrega de gestionar-les de manera òptima. Si un contenidor falla, el sistema el reemplaça automàticament. Si no hi ha recursos suficients per executar tots els contenidors, Kubernetes espera fins que els recursos estiguin disponibles per complir els requisits especificats.

Un dels aspectes més destacats de Kubernetes és el seu ecosistema. Ha esdevingut el principal estàndard per al desplegament d'aplicacions al núvol, i moltes eines compten amb versions dissenyades específicament per funcionar-hi. A més, Kubernetes ha fet realitat el concepte de multi-cloud: empaquetant una aplicació per Kubernetes, es pot desplegar fàcilment tant en AWS, Azure, Google Cloud com en un clúster local (on-premises). Aquesta capacitat és essencial en entorns moderns de TI per garantir flexibilitat i escalabilitat.

Kubernetes distribueix automàticament el tràfic entre les rèpliques dels contenidors, evitant sobrecàrregues. Si un contenidor falla, Kubernetes el substitueix automàticament sense interrupcions significatives al servei. A més porta de caixa un aïllament d'entorns que facilita la gestió d'entorns de desenvolupament, prova i producció, assegurant coherència entre ells.

Arquitectura i components de Kubernetes

Kubernetes divideix els nodes d'un clúster en master nodes i worker nodes.

- Els master nodes constitueixen la capa de control i, en principi, no executen contenidors d'usuari.
- Els worker nodes són responsables d'executar els contenidors d'usuari.

Tot i que aquesta divisió és l'habitual, quan Kubernetes s'executa localment,

sovint es crea un clúster d'un sol node que actua tant com a master com a worker node.

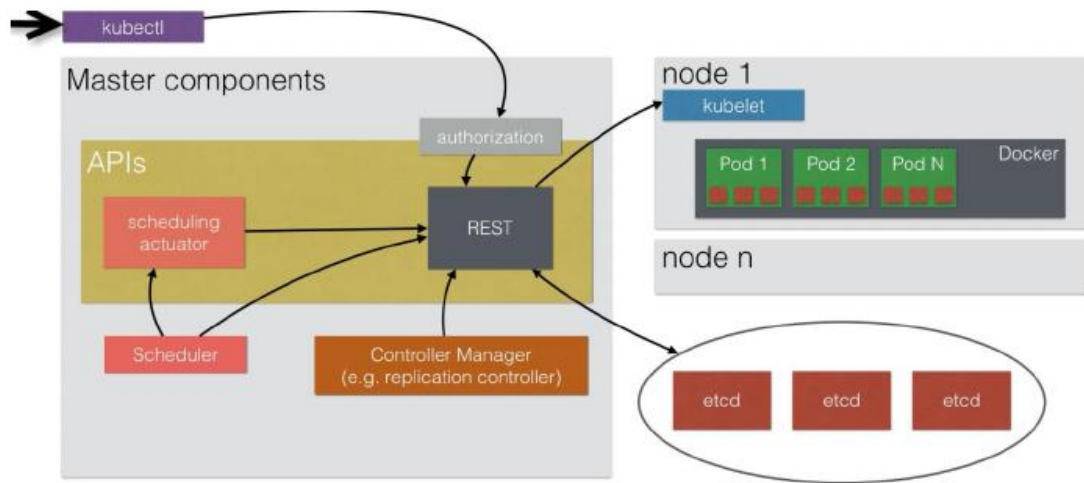


Figura 13. Diagrama Arquitectura Kubernetes

La lògica de la capa de control es basa en `etcd`, una base de dades distribuïda que garanteix la consistència de les dades utilitzant el protocol RAFT. Sobre aquesta base de dades, s'implementa una capa REST que regula tots els accessos a `etcd`. Aquesta capa REST s'exposa a través d'un mòdul d'autenticació, cosa que permet interactuar-hi amb eines com els comandaments `kubectll`.

A més, la lògica que s'executa als master nodes inclou diversos controladors. Aquests són funcions que s'executen en bucle per assegurar que l'estat desitjat del clúster coincideix amb l'estat actual. Si no coincideixen, els controladors s'encarreguen de crear o eliminar contenidors segons sigui necessari.

Normalment, els masters són sempre un nombre imparell de nodes, sovint tres. En clústers més grans, es pot escalar a cinc nodes per garantir una gestió més robusta i una alta disponibilitat.

Els worker nodes, a més dels contenidors d'usuari, inclouen principalment dos components essencials: el `kubelet` i el `kube-proxy`.

El `kubelet` té la funció de verificar l'estat de `etcd` a través de la capa REST per determinar quins contenidors se li han assignat per executar. A partir d'aquesta informació, s'encarrega de crear o eliminar contenidors en el node que gestiona.

Per la seva banda, el `kube-proxy` és responsable de gestionar la xarxa i el service discovery entre contenidors. Això ho fa mitjançant la creació de regles a `iptables`, assegurant que els contenidors puguin comunicar-se de manera eficient dins del clúster.

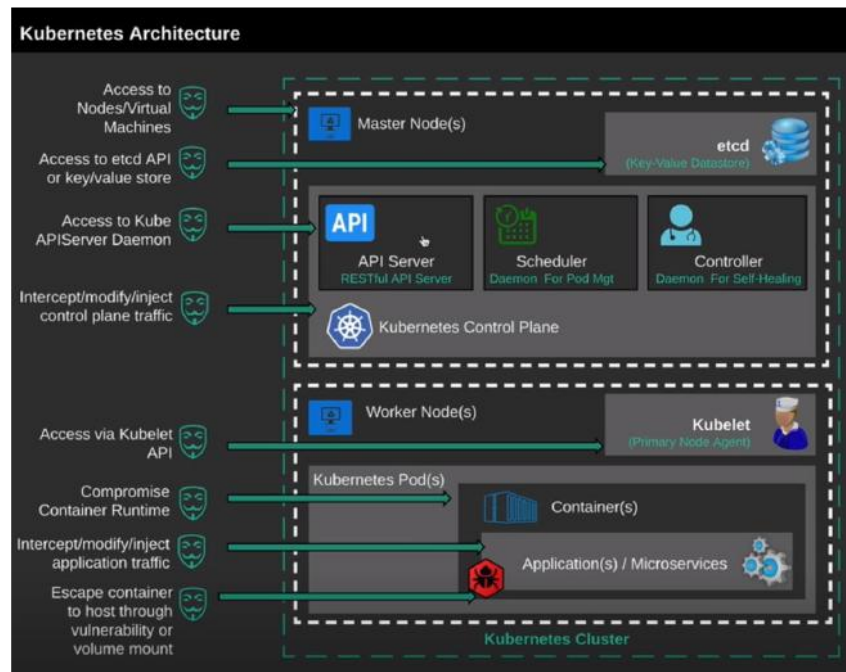
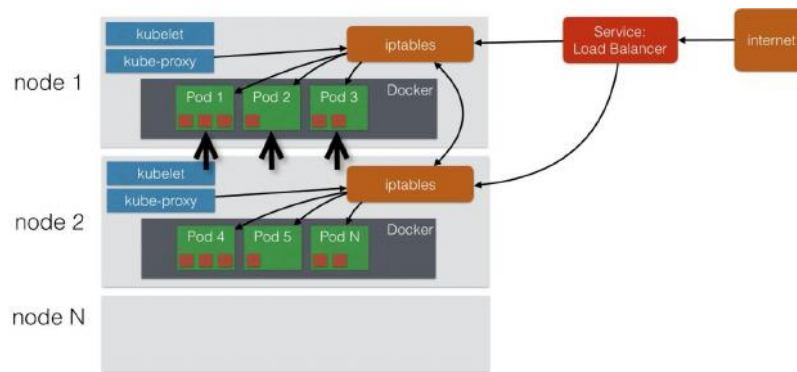


Figura 14. Diagrama detallat Arquitectura Kubernetes

Integració Contenedors-Orquestració en el Projecte

En la descomposició inicial del monòlit, identifiquem components del monòlit que es poden separar en microserveis, on cada component es desenvolupa com un contenidor Docker.

Utilitzem Kubernetes per gestionar les diferents fases del desplegament, assegurant que els microserveis nous funcionin correctament i que els usuaris no experimentin interrupcions. En encapsular els serveis en contenidors i gestionar-los amb Kubernetes, podem desplegar funcionalitats més ràpidament sense afectar altres serveis.

Per un altre costat, Kubernetes facilita la gestió de dependències entre microserveis (com l'ús de bases de dades compartides), mitjançant configuracions declaratives i sistemes de xarxa interna.

A mesura que la demanda d'un microservei augmenta, Kubernetes escala les instàncies només d'aquest servei, optimitzant l'ús de recursos. L'ús de Kubernetes permet fer ús òptim de la infraestructura, reduint els costos associats al consum energètic i a la sobredimensió d'entorns monolítics i addicionalment garanteix un aïllament dels contenidors que minimitza l'impacte d'errors locals.

Aquest enfocament permet una fàcil migració a entorns de núvol híbrid o totalment basats en núvol, maximitzant la flexibilitat. Aconseguint doncs amb aquesta estratègia fer una transició des d'una arquitectura monolítica a microserveis d'una manera controlada, eficient i escalable.

2.5 Integració i Desplegament Continus

En el nostre projecte de migració d'una arquitectura monolítica a microserveis, és fonamental adoptar eines d'integració i desplegament continu (CI/CD) per garantir una transició eficient i sense errors. En aquest context, GitLab Pipelines, HELM i ArgoCD es converteixen en els pilars principals per automatitzar i gestionar tant el procés de desenvolupament com el de desplegament dels microserveis.

En una aplicació monolítica, cada canvi en el codi requereix reconstruir i desplegar tota l'aplicació, fet que pot ser lent i propens a errors. Quan es fa la migració a microserveis, cada component es converteix en un servei independent, el que fa que la gestió d'aquests serveis i els seus desplegaments sigui més complexa. Per resoldre aquest repte, necessitem eines que facilitin la integració i el desplegament dels microserveis de manera automàtica, sincronitzant l'estat de la infraestructura amb les configuracions desitjades i garantint la traçabilitat dels canvis.

GitLab Pipelines ajuda a automatitzar el procés de desenvolupament de microserveis. Per cada microservei, es crea una pipeline independent que facilita la construcció, proves i desplegament de manera modular. A través de Docker, els microserveis es despleguen com imatges contenidors lleugeres i escalables, el que facilita l'automatització del procés i la seva validació abans de la implementació en producció. A més, el procés de CI/CD ajuda a reduir l'impacte dels errors i millorar la qualitat dels canvis en el codi.

HELM i ArgoCD: Implementació de GitOps per a la Gestió del Desplegament

Un cop els microserveis estan creats i empaquetats amb Docker, ArgoCD permet gestionar el desplegament d'aquests microserveis en un entorn Kubernetes. Mitjançant el mètode GitOps, ArgoCD sincronitza el clúster de Kubernetes amb els canvis emmagatzemats en un repositori Git, garantint que el sistema sempre estigui alineat amb les configuracions declarades en el repositori. ArgoCD també facilita la implementació de estratègies de desplegament controlades, com les blue/green deployments o les canary releases, que permeten desplegar noves versions dels serveis de manera gradual, minimitzant els riscos en producció. [\[7\]](#)

Aquesta gestió es veu facilitada per l'ús de **Helm**, una eina de gestió de paquets per a Kubernetes. Helm utilitza "charts", que són plantilles preconfigurades que faciliten el desplegament i la gestió de les aplicacions a Kubernetes. Mitjançant Helm, es poden definir fàcilment els components d'una aplicació Kubernetes (com el front-end, el back-end, bases de dades, etc.) i configurar-los de manera modular i reutilitzable. Això és especialment útil per a una arquitectura de microserveis, on es poden gestionar diversos components d'una manera centralitzada i amb una configuració consistent a través de l'ús de variables globals per a cada entorn.

Per exemple, els **Helm charts** poden ser configurats per definir els paràmetres de desplegament de cada component de l'aplicació: front-end, back-end, base de dades, etc., amb fitxers de configuració globals que permeten modificar variables clau com el producte que s'està desplegant, els serveis específics o els microserveis implicats, i fins i tot l'entorn (producció, desenvolupament, etc.). Això facilita una gran flexibilitat i reutilització de codi entre els diversos serveis, ja que només cal modificar el fitxer de configuració per a canviar el comportament d'un servei específic o d'una aplicació sencera.

El flux entre **GitLab Pipelines** i **ArgoCD** comença quan un desenvolupador fa un canvi en el codi del microservei i el puja al repositori Git. Des d'allà, GitLab executa el procés de CI, que inclou la compilació del codi, l'execució de proves i la creació d'una imatge Docker del microservei. Un cop la imatge es publica en el registre de contenidors, ArgoCD detecta el canvi i sincronitza el desplegament del microservei a Kubernetes, actualitzant l'entorn de prova o producció segons sigui necessari.

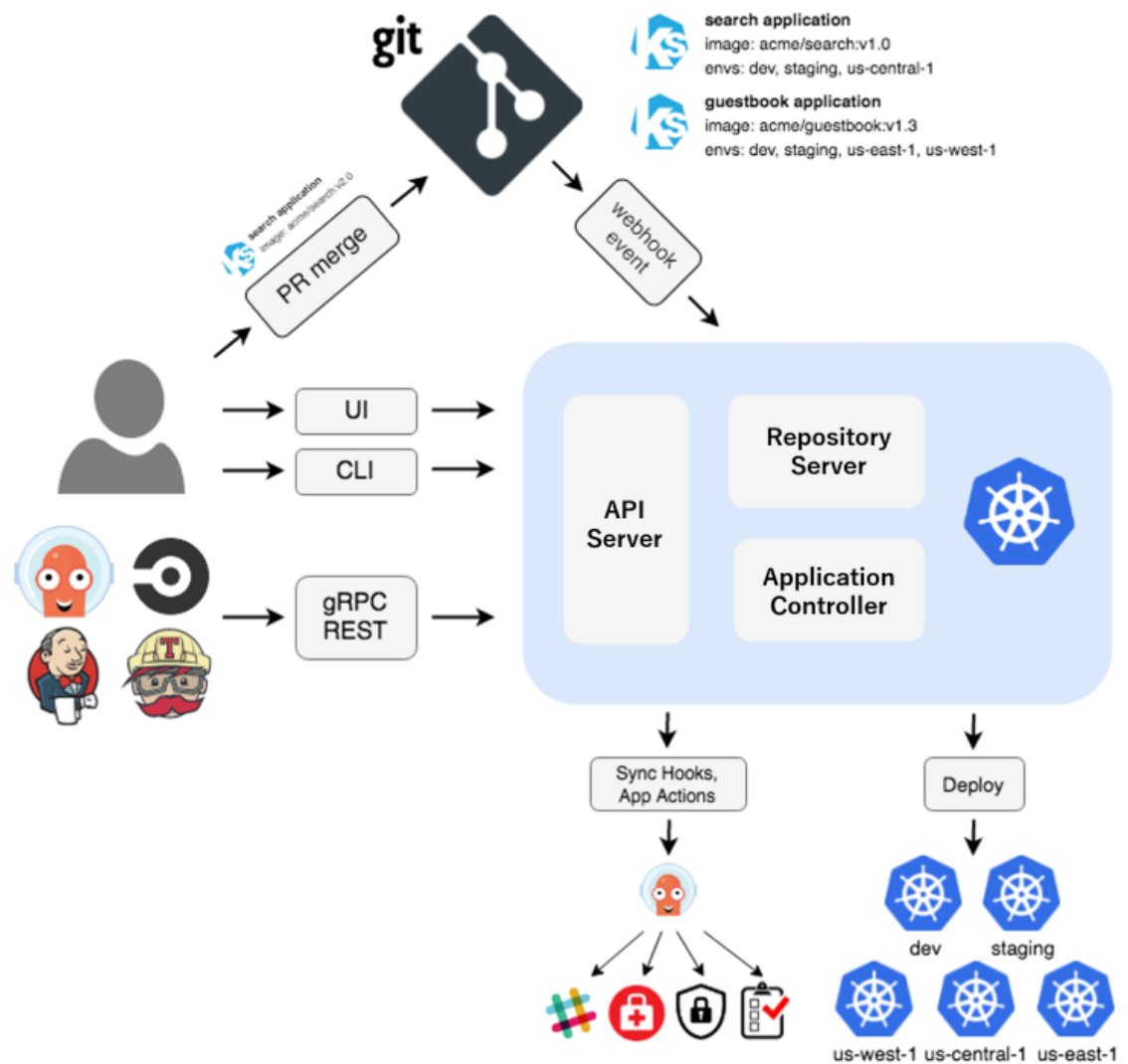


Figura 15. Diagrama Arquitectura ArgoCD- <https://argo-cd.readthedocs.io/en/stable/>

Aquest enfocament permet diversos avantatges clau en el context de la migració d'una arquitectura monolítica a microserveis. Cada microservei es pot desplegar de manera independent, reduint els temps d'inactivitat i millorant la disponibilitat general del sistema. A més, es poden escalar de manera independent, permetent adaptar-se millor a les fluctuacions de la demanda de recursos.

Automatitzant el procés de proves i desplegament, es redueix el temps necessari per implementar canvis i es minimitzen els errors humans. Així doncs, gràcies a eines de metodologia GitOps com ArgoCD i Helm, els canvis en la configuració de l'aplicació es poden reflectir immediatament en la infraestructura, millorant l'agilitat i la capacitat de resposta davant de noves necessitats.

3. Disseny del Sistema

3.1 Identificació de Components

Per abordar la migració, el primer pas és descompondre l'aplicació monolítica en components autònoms que es converteixin en microserveis. Aquesta descomposició s'ha dut a terme mitjançant una anàlisi detallada dels mòduls existents, la seva funcionalitat, i les seves interdependències.

Estratègia d'Identificació

Anàlisi Funcional

L'anàlisi funcional és una etapa clau en la migració des d'una arquitectura monolítica a una basada en microserveis. En aquest procés, es desglossa l'aplicació existent en funcions independents, identificant-ne els requisits específics i establint quines d'elles poden convertir-se en microserveis autònoms.

Objectiu de l'Anàlisi Funcional

L'objectiu principal és determinar les funcionalitats crítiques de l'aplicació, la seva relació amb altres components, i com aquestes poden ser desacoblades. Això permet assegurar que cada microservei tindrà una única responsabilitat, SRP (Single Responsibility Principle), facilitant la seva gestió i desplegament independent.

Procés de l'Anàlisi Funcional

1. **Identificació de les Funcionalitats Clau:**
 - Es revisa l'aplicació monolítica per identificar els blocs funcionals principals, com ara autenticació, gestió d'usuaris, processament de pagaments, o generació de informes.
 - Cada funcionalitat s'analitza per determinar-ne la complexitat i dependències.
2. **Mapa d'Interaccions i Dependències:**
 - Es crea un mapa que mostra com interactuen els mòduls entre si i amb altres serveis o bases de dades.
 - Exemple: Es pot identificar que el mòdul de facturació depèn tant de la gestió de clients com de l'inventari.
3. **Avaluació d'Autonomia:**
 - S'analitza quines funcions poden operar de manera independent sense afectar les altres. Aquest pas determina quines funcionalitats són candidates a convertir-se en microserveis.
4. **Classificació en Domini de Negoci:**
 - Les funcionalitats s'organitzen segons les àrees de negoci a les quals pertanyen, com ara serveis per a clients, operacions internes, o integracions amb tercers.

Resultats de l'Anàlisi Funcional

1. Funcions Clau Identificades:

- **Gestió d'usuaris:** Inclou registres, autenticació i permisos.
- **Processament d'ordres:** Gestió de comandes, pagaments i facturació.
- **Anàlisi i informes:** Generació de mètriques i visualitzacions per a l'usuari o el negoci.
- **Notificacions:** Enviament de correus electrònics o missatges push als usuaris.

2. Reorganització dels Components:

- Cada funcionalitat es defineix com un microservei independent, amb el seu propi repositori de codi, pipeline de CI/CD i base de dades.

Beneficis de l'Anàlisi Funcional

- Redueix la complexitat en el desenvolupament i manteniment.
- Facilita l'escalabilitat perquè cada microservei pot créixer de manera independent.
- Millora la tolerància a fallades, ja que problemes en un microservei no afecten la resta del sistema.

Aquest procés és fonamental per establir les bases de l'arquitectura final i garantir que la transició cap als microserveis sigui eficient i alineada amb els objectius del projecte.

Domini de Negoci i Agrupació Funcional

En una arquitectura basada en microserveis, és fonamental organitzar els serveis al voltant dels dominis de negoci del sistema. Aquesta agrupació permet que cada microservei se centri en una àrea específica del negoci, reduint les dependències i millorant la modularitat. Aquesta estratègia segueix els principis del Domain-Driven Design (DDD) [\[8\]](#), que busca reflectir la lògica del negoci dins del disseny tècnic del sistema.

En el context del disseny d'un sistema basat en microserveis, el concepte de domini de negoci i agrupació funcional és essencial per estructurar i organitzar els components del sistema de manera coherent amb les necessitats empresarials.

El domini de negoci es refereix a les àrees específiques dins d'una organització que tenen funcions o propòsits clarament definits. Per exemple, en una empresa de comerç electrònic, els dominis de negoci poden incloure la gestió de clients, processament de pagaments, inventari de productes o logística d'enviaments. Identificar aquests dominis permet entendre com s'organitzen les operacions i quines funcionalitats són crucials per al negoci.

L'agrupació funcional, d'altra banda, consisteix a dividir i assignar les diferents funcionalitats del sistema a aquests dominis de negoci. Això assegura que cada component o microservei s'encarrega d'una tasca específica i alineada amb el domini al qual pertany. Per exemple, el servei de gestió d'usuaris podria ser

responsable del registre, autenticació i administració de dades personals, mentre que el servei de pagaments gestionaria processos relacionats amb targetes de crèdit o passarel·les de pagament.

Aquest enfocament aporta diversos beneficis:

Claredat i coherència en el sistema: Quan cada microservei correspon a un domini de negoci concret, es redueix la complexitat, ja que les responsabilitats estan ben delimitades i alineades amb les operacions empresarials.

Escalabilitat i modularitat: Els dominis de negoci permeten que els microserveis siguin fàcilment escalables i modificables sense afectar altres parts del sistema. Per exemple, si hi ha un augment en les sol·licituds al servei de pagaments, només aquest servei necessitarà més recursos.

Millora en la col·laboració entre equips: Agrupar funcionalitats segons dominis facilita que els equips de desenvolupament i gestió puguin treballar amb més autonomia i concentrar-se en àrees específiques del negoci, en lloc d'haver d'entendre tot el sistema.

Aquest procés també ajuda a identificar interdependències entre dominis, com les interaccions entre logística i inventari o entre vendes i processament de dades de clients. Això és essencial per assegurar que els microserveis treballen de manera coordinada sense crear dependències innecessàries que puguin complicar el sistema.

Al nostre agrupament per domini funcional hem de tenir en compte:

1. Gestió de Clients

Inclou tots els serveis relacionats amb els usuaris finals, com ara:

- **Registre i Autenticació:** Serveis per gestionar l'alta d'usuaris, inici de sessió i control d'accessos.
- **Perfil d'Usuari:** Gestió de la informació personal, preferències i configuracions.
- **Notificacions:** Sistemes per enviar correus electrònics, SMS o notificacions push.

2. Processament d'Ordres

Responsable de gestionar els processos centrals del negoci:

- **Gestió de Comandes:** Creació, seguiment i actualització de les comandes.
- **Pagaments i Facturació:** Integració amb passarel·les de pagament, càlcul de factures i generació de comprovants.
- **Inventari:** Seguiment d'existències i disponibilitat de productes.

3. Anàlisi i Informes

Aquest domini està enfocat en extreure valor de les dades:

- **Generació d'Informes:** Estadístiques d'ús, mètriques financeres i visualitzacions personalitzades.

- **Anàlisi Predictiva:** Models per anticipar comportaments o tendències basats en dades històriques.
4. **Integracions Externes**
- S'encarrega de la comunicació amb sistemes i serveis de tercers:
- **Integracions API:** Connexió amb proveïdors, socis comercials o plataformes externes.
 - **Automatitzacions:** Eines per sincronitzar dades amb sistemes externs en temps real o de manera programada.

Aquest enfocament garanteix que la nova arquitectura estigui no només tècnicament optimitzada sinó també profundament alineada amb els objectius i requisits estratègics del negoci.

Desacoblament de Dependències

Un cop ben definit el domini funcional de la nostra aplicació monolítica, cal començar a treballar en el desacoblament de dependències de la mateixa, doncs és un dels principis fonamentals de la migració d'una arquitectura monolítica a una basada en microserveis. Aquest procés busca reduir les interdependències entre els components d'un sistema, fent que cada microservei sigui autònom i pugui funcionar independentment dels altres. Aquesta estratègia no només millora la modularitat sinó que també facilita el manteniment, l'actualització i l'escalabilitat del sistema.

El principal objectiu és trencar amb les fortes dependències inherents a una arquitectura monolítica, on un canvi en un mòdul pot implicar canvis o errors en altres parts del sistema. En un entorn de microserveis:

- Cada servei pot ser desenvolupat, desplegat i mantingut independentment.
- Les actualitzacions o fallades en un servei no han d'afectar el conjunt del sistema.
- Els equips de treball poden gestionar serveis de manera autònoma, assignant responsabilitats per domini funcional.

Estratègies per al Desacoblament

Separació de lògica de negoci: Identificar els components clau del sistema i reassignar-los a serveis independents. Això implica agrupar funcionalitats relacionades dins d'un microservei únic.

Implementació d'APIs: Les APIs serveixen com a interfície estàndard per permetre la comunicació entre serveis sense establir connexions directes ni dependències fortes.

Bases de dades descentralitzades: Cada microservei hauria de gestionar les seves pròpies dades per evitar colls d'ampolla i dependències en un esquema de base de dades monolític compartit.

Asincronia: Utilitzar sistemes de missatgeria (com Kafka o RabbitMQ) per facilitar la comunicació entre serveis sense necessitat d'una dependència sincrònica.

Avantatges del Desacoblament

El desacoblament és un concepte clau en el disseny d'una arquitectura de microserveis, que aporta nombrosos avantatges en termes de gestió, escalabilitat i resiliència del sistema. A diferència de les arquitectures monolítiques, on els components depenen estretament els uns dels altres, en una arquitectura desacoblada cada servei o component pot funcionar de manera autònoma, la qual cosa comporta múltiples beneficis.

Un dels avantatges més significatius és la flexibilitat en el desenvolupament i manteniment del sistema. Quan els components estan desacoblats, és possible fer modificacions o actualitzacions en una part del sistema sense haver d'alterar altres components. Això resulta especialment útil per introduir noves funcionalitats o corregir errors sense risc de causar problemes en tot el sistema, cosa que és molt més complicada en un entorn monolític.

També millora la resiliència i disponibilitat del sistema. Si un component falla, el seu impacte pot limitar-se únicament a aquella funcionalitat, mentre que la resta del sistema continua operant amb normalitat. Això és essencial per a aplicacions crítiques on la interrupció de serveis pot tenir conseqüències greus.

El desacoblament afavoreix la escalabilitat modular. En lloc d'escalar tot el sistema, podem centrar-nos en els components específics que experimenten una càrrega elevada. Per exemple, un servei encarregat de processar pagaments pot ser replicat per gestionar més transaccions, sense necessitat d'escalar altres serveis no afectats.

Un altre aspecte clau és la independència tecnològica. Cada component pot utilitzar les eines, llenguatges de programació o bases de dades més adequades per a la seva funció específica. Això permet optimitzar recursos i aprofitar les millors tecnologies disponibles per a cada cas.

Finalment, el desacoblament fomenta una cultura d'autonomia en els equips de desenvolupament. Cada equip pot treballar en un servei independent, accelerant el desenvolupament i millorant la col·laboració. Aquesta independència també redueix les dependències entre equips, facilitant l'adopció de metodologies àgils com DevOps o Scrum.

En conjunt, el desacoblament no només millora el rendiment tècnic del sistema, sinó que també contribueix a una gestió més eficient i escalable, alineada amb les necessitats d'un entorn de negoci canviant i competitiu.

Components Identificats

Durant la fase d'identificació de components en el procés de migració a una arquitectura de microserveis, hem posat el focus en desglossar el sistema en unitats independents però coordinades que puguin operar de manera autònoma. Aquesta tasca és fonamental per aconseguir els objectius de modularitat, escalabilitat i flexibilitat que defineixen una arquitectura basada en microserveis.

- Front-End (Capa de Presentació)
- Back-End (Capa de Lògica de Negoci)
- Base de Dades (Persistència)
- Middleware i Comunicació
- Gestió d'APIs
- Operació i Gestió del Sistema
- Monitorització i Observabilitat
- Backups i Contingència davant Desastres
- Seguretat

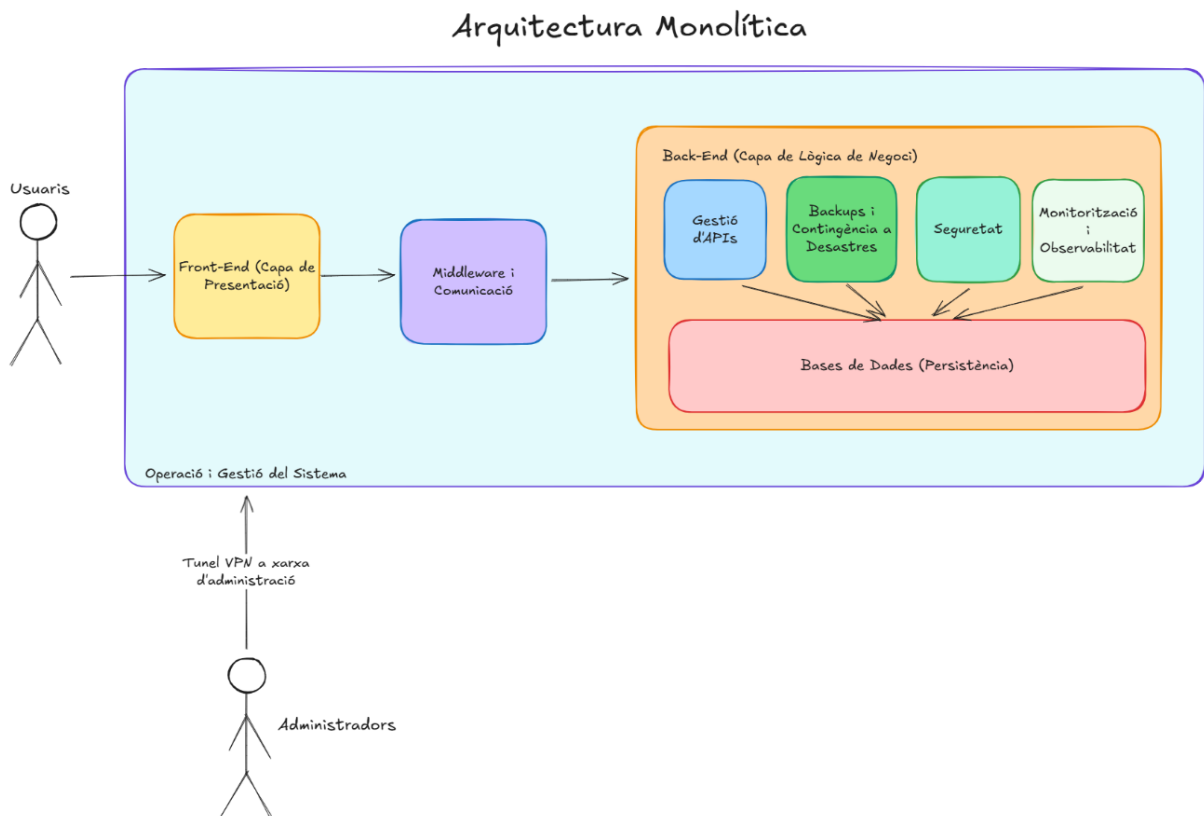


Figura 16. Diagrama alt nivell de components funcionals en una Arquitectura Monolítica. Creació pròpia

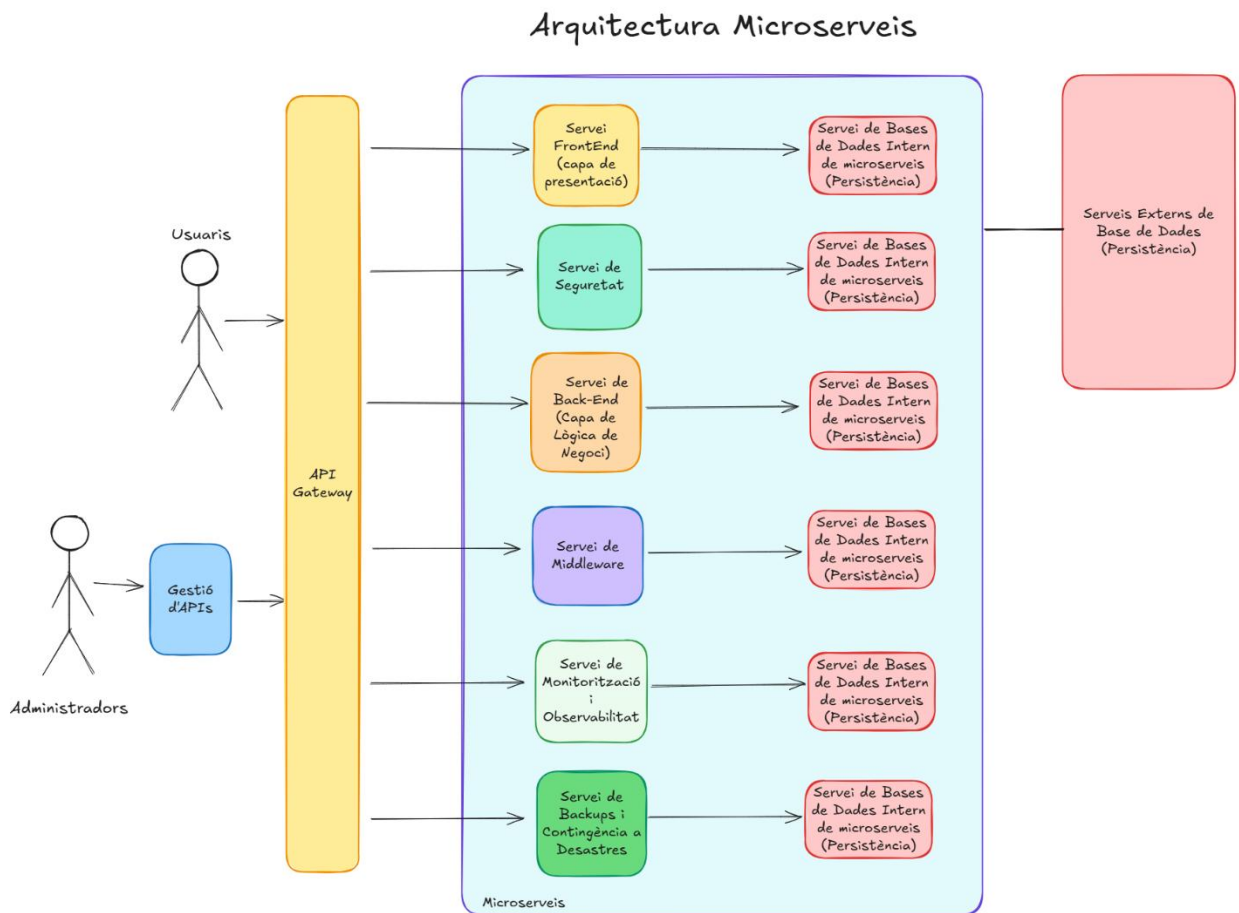


Figura 17. Diagrama alt nivell de components funcionals en una Arquitectura de Microserveis. Creació pròpia

Un dels primers components identificats és el **front-end** o capa de presentació. Aquest s'encarregarà de gestionar la interacció amb els usuaris, proporcionant una interfície clara i funcional. Aquí, tecnologies modernes com React o Angular poden jugar un paper crucial per oferir experiències intuïtives i compatibles amb múltiples dispositius. El repte principal serà assegurar una comunicació fluida amb les API del back-end, especialment per garantir actualitzacions en temps real.^[9]

El **back-end** és el motor del sistema i, en el context d'una arquitectura de microserveis, es divideix en serveis especialitzats. Per exemple, podem tenir un servei per gestionar els usuaris i les seves dades, un altre per administrar recursos o productes, i un tercer per controlar processos com els pagaments. Aquesta fragmentació permet desenvolupar, desplegar i mantenir cada servei independentment, un avantatge clau sobre les arquitectures monolítiques.

Pel que fa a la **persistència de dades**, hem considerat solucions tant relacionals com NoSQL, segons les necessitats de cada servei. Per a dades estructurades i transaccionals, PostgreSQL és una opció robusta, mentre que MongoDB pot ser útil per gestionar dades més flexibles o grans volums que necessiten escalabilitat. La comunicació entre microserveis és un altre element crític. Aquesta es gestionarà a través d'un **middleware**, que assegni la transmissió eficient i segura

d'informació entre serveis. Aquí es poden utilitzar tecnologies com RabbitMQ o Kafka per a la comunicació asíncrona, i REST o gRPC per a la comunicació directa.

En un sistema basat en microserveis, la gestió de les **APIs** es converteix en un component estratègic. Les APIs actuen com a portes d'entrada per accedir als diferents serveis i faciliten la integració amb altres sistemes. Implementar un API Gateway robust, com Kong, ajudarà a gestionar l'accés, monitoritzar l'ús i garantir la seguretat.

En el disseny d'una arquitectura basada en microserveis, la inclusió de mecanismes de **backup i contingència davant desastres** és fonamental per assegurar la fiabilitat i la continuïtat del servei. Aquests components no només serveixen per protegir les dades i els sistemes, sinó també per garantir que, en cas d'incident, el sistema pugui recuperar-se ràpidament amb el menor impacte possible.

El **sistema de backup** es dissenya per capturar regularment còpies de seguretat de les dades clau, així com dels estats crítics dels serveis. Aquest procés es realitza de manera automàtica i freqüent, utilitzant eines compatibles amb entorns distribuïts i contenidors. Un altre aspecte essencial és la seguretat d'aquestes còpies, que es xifren per complir amb normatives com el RGPD, protegint les dades contra accessos no autoritzats. A més, les còpies es distribueixen geogràficament per garantir la seva disponibilitat fins i tot en cas de fallades locals.

Per altra banda, el **component de contingència davant desastres** es centra a establir mecanismes que permetin la recuperació ràpida del sistema. Això inclou la configuració d'entorns de rèplica en zones diferents, els quals poden assumir el trànsit de manera immediata si es produeix una caiguda en el servei principal. A més, eines d'orquestració com Kubernetes poden reiniciar serveis automàticament en cas de fallada, mentre que eines d'observabilitat com Prometheus anticipen possibles problemes, facilitant una resposta proactiva.[\[10\]](#)

Un aspecte clau d'aquests sistemes és la seva integració amb els microserveis. Això permet que cada servei gestioni els seus propis backups i participi en els plans de recuperació, creant una xarxa resiliència i coordinada que reforça la robustesa general del sistema. Aquests mecanismes són essencials per mantenir la confiança dels usuaris i reduir els impactes dels incidents imprevistos.

En un sistema basat en microserveis, la **monitorització** tradicional és essencial per conèixer l'estat general del sistema, però sovint es queda curta per comprendre les interaccions complexes entre serveis. Aquí entra en joc el concepte d'**observabilitat**, que va més enllà de recollir mètriques i es centra en proporcionar una visió completa i detallada del comportament del sistema a través de tres pilars principals:

- **Mètriques:** Informació quantitativa sobre el rendiment, com ara l'ús de recursos, temps de resposta o càrregues dels serveis.

- **Traçabilitat:** Registres que segueixen les peticions a través dels diferents serveis, permetent identificar colls d'ampolla o punts de fallada.
- **Logs:** Històrics detallats d'esdeveniments i errors, imprescindibles per analitzar problemes específics.

Aquest enfocament d'observabilitat no només ajuda a detectar problemes ràpidament, sinó que també facilita l'anàlisi proactiva de dades per anticipar possibles incidències abans que es converteixin en problemes crònics.

Per implementar aquesta capacitat, s'utilitzen eines com **Prometheus** i **Grafana**, que recopilen i visualitzen mètriques de rendiment, o **Jaeger** i **OpenTelemetry**, que són ideals per gestionar la traçabilitat. Aquesta combinació permet obtenir una visió completa, fins i tot en arquitectures distribuïdes complexes. [\[11\]](#)

Un altre aspecte fonamental en un sistema de microserveis és garantir una operació i gestió eficient del sistema en entorns de producció. Aquesta gestió implica diversos processos clau:

Automatització de les operacions: Les eines d'orquestració, com Kubernetes, juguen un paper essencial a l'hora de gestionar la distribució, escalabilitat i recuperació automàtica dels serveis. Aquestes operacions es realitzen de manera contínua i automàtica, evitant interrupcions del servei.

- **Gestió de configuracions:** Eines com **Helm** permeten gestionar configuracions complexes per a múltiples serveis de manera coherent. Això assegura que els canvis en els paràmetres d'un servei no interfereixin amb els altres.
- **Còpies de seguretat i recuperació:** Els processos de backup regulars i els plans de recuperació davant desastres (ja descrits) són també una part integral de la gestió del sistema.
- **Gestió de recursos:** En entorns dinàmics, com els núvols híbrids, és important optimitzar l'ús dels recursos. Aquesta gestió inclou ajustar l'escala dels serveis segons la demanda, garantint un ús eficient de les infraestructures disponibles.
- **Documentació i formació:** Perquè la gestió del sistema sigui sostenible a llarg termini, és imprescindible mantenir documentats tots els processos i assegurar que l'equip d'operacions rebi formació continuada sobre les eines i les pràctiques més actualitzades.

Amb una operació ben gestionada, es pot garantir que el sistema de microserveis es mantingui no només funcional, sinó també flexible i capaç d'adaptar-se a necessitats futures sense comprometre la qualitat o la disponibilitat del servei.

Continuant en l'aspecte de configuració i operació de la plataforma, una peça clau és la gestió del codi amb HELM, ArgoCD i GitLab Pipelines és una combinació potent que permet una gestió més eficient i fiable de microserveis dins d'una arquitectura moderna. Aquest enfocament s'emmarca en els principis de **GitOps**, que utilitza repositoris Git per definir l'estat desitjat tant de les aplicacions com de

la infraestructura. Aquesta metodologia no només facilita la traçabilitat dels canvis, sinó que també assegura que la infraestructura està sempre sincronitzada amb les configuracions definides.

Helm, com a eina de gestió de paquets per a Kubernetes, facilita encara més aquest flux. Mitjançant **Helm charts**, es poden empaquetar i gestionar les aplicacions Kubernetes de manera modular i reutilitzable. Això permet configurar fàcilment els components de l'aplicació com el front-end, el back-end o les bases de dades. Els **Helm charts** inclouen plantilles que definixen els paràmetres de desplegament per a cadascun d'aquests components, amb fitxers de configuració globals que permeten modificar variables clau com el producte que s'està desplegant, els serveis específics, l'entorn (producció, desenvolupament, etc.) i altres aspectes del sistema, tot mantenint una coherència i flexibilitat a través de l'ús de configuracions reutilitzables.

ArgoCD, en particular, destaca per la seva capacitat per gestionar el desplegament d'aplicacions dins de Kubernetes. Una de les seves característiques més interessants és el patró "**App of Apps**", on una aplicació principal conté referències a altres aplicacions o microserveis que s'administren de manera independent. **Aquesta estratègia permet gestionar entorns complexos amb desenes o fins i tot centenars de microserveis sense perdre el control ni la visibilitat.** Tot està centralitzat en un repositori Git, i qualsevol canvi que es faci en aquest repositori es pot aplicar automàticament al clúster Kubernetes.

D'altra banda, **GitLab Pipelines** complementa aquesta capacitat gestionant tot el procés de desenvolupament i integració dels microserveis. Cada pipeline pot incloure etapes com la compilació, les proves, la creació d'imatges Docker i la seva publicació a un registre de contenidors. Això garanteix que **cada canvi al codi es validi i es prepari adequadament abans del desplegament.** Una vegada publicat el codi o la configuració actualitzada, ArgoCD detecta automàticament aquests canvis i s'encarrega d'aplicar-los al clúster, tancant així el cercle d'automatització.

Aquest enfocament permet desplegar canvis ràpidament, reduir el risc d'errors, i millorar la traçabilitat i l'auditoria del sistema. Per exemple, quan s'actualitza un microservei, només aquest component es desplega, sense afectar la resta del sistema, la qual cosa és fonamental en un entorn de microserveis. A més, amb ArgoCD es poden implementar estratègies avançades com els *canary deployments* o els desplegaments *blue/green*, augmentant encara més la robustesa dels desplegaments.

Aquesta combinació d'eines, amb **HELM** per gestionar la configuració de l'aplicació, **ArgoCD** per orquestrar el desplegament i **GitLab Pipelines** per a la integració contínua, és essencial per garantir que els components identificats, com el Front-End, el Back-End, o la Gestió d'APIs, es despleguin de manera sincronitzada i coherent amb la infraestructura. [\[12\]](#).

Exemple de pipeline:

```
stages:
  - build
  - test
  - deploy
build_job:
  stage: build
  script:
    - docker build -t $CI_REGISTRY_IMAGE:$CI_COMMIT_SHORT_SHA .
    - docker push $CI_REGISTRY_IMAGE:$CI_COMMIT_SHORT_SHA
deploy_job:
  stage: deploy
  script:
    - kubectl apply -f k8s/
```

Exemple de Helm Chart:

```
myapp/
├── Chart.yaml
├── values.yaml
├── charts/
├── templates/
│   ├── deployment.yaml
│   ├── service.yaml
│   └── ingress.yaml
```

1. Chart.yaml

Aquest fitxer conté la informació bàsica del Helm chart.

```
apiVersion: v2
name: myapp
description: A simple web application
version: 0.1.0
appVersion: "1.0"
```

2. values.yaml

Aquest fitxer defineix les variables configurables per a l'aplicació i l'entorn. Permet personalitzar l'estructura del desplegament segons l'entorn (producció, desenvolupament, etc.).

```
replicaCount: 1

image:
  repository: myapp-image
  tag: latest
  pullPolicy: IfNotPresent

service:
  name: myapp-service
  type: ClusterIP
  port: 80

ingress:
```

```

enabled: false
path: /
hostname: myapp.local

resources: {}

```

3. templates/deployment.yaml

Aquest fitxer defineix el desplegament de l'aplicació, com el nombre de rèpliques i el contenidor Docker que s'ha d'utilitzar.

```

apiVersion: apps/v1
kind: Deployment
metadata:
  name: {{ include "myapp.fullname" . }}
  labels:
    app: {{ include "myapp.name" . }}
spec:
  replicas: {{ .Values.replicaCount }}
  selector:
    matchLabels:
      app: {{ include "myapp.name" . }}
  template:
    metadata:
      labels:
        app: {{ include "myapp.name" . }}
    spec:
      containers:
        - name: myapp
          image:
            "{{ .Values.image.repository }}:{{ .Values.image.tag }}"
          ports:
            - containerPort: 80

```

4. templates/service.yaml

Aquest fitxer defineix un servei per a l'aplicació per permetre que altres components dins de Kubernetes puguin comunicar-se amb ella.

```

apiVersion: v1
kind: Service
metadata:
  name: {{ include "myapp.fullname" . }}
spec:
  ports:
    - port: {{ .Values.service.port }}
      targetPort: 80
  selector:
    app: {{ include "myapp.name" . }}

```

5. templates/ingress.yaml

Si es vol exposar l'aplicació a l'exterior (per exemple, mitjançant un controlador d'ingress), es pot utilitzar aquest fitxer:

```

apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: {{ include "myapp.fullname" . }}
spec:
  rules:
  - host: {{ .Values.ingress.hostname }}
    http:
      paths:
      - path: {{ .Values.ingress.path }}
        pathType: Prefix
        backend:
          service:
            name: {{ include "myapp.fullname" . }}
            port:
              number: 80

```

Beneficis combinats d'ArgoCD, HELM i GitLab Pipelines

- **Automatització:** ArgoCD redueix la necessitat d'intervencions manuals al desplegament. GitLab Pipelines s'encarrega de la creació i verificació de codi de forma automatitzada.
- **Velocitat i fiabilitat:** En un entorn de microserveis, aquests dos components treballen junts per assegurar que els canvis es despleguin ràpidament amb una alta fiabilitat.
- **Adaptació contínua:** Les configuracions poden ser modificades als HELM charts fàcilment i ArgoCD sincronitza els canvis immediatament. Aquest enfocament facilita la gestió d'entorns complexos, assegurant que els components identificats (com Front-End, Back-End, Middleware, i altres) es mantenen consistents i actualitzats.

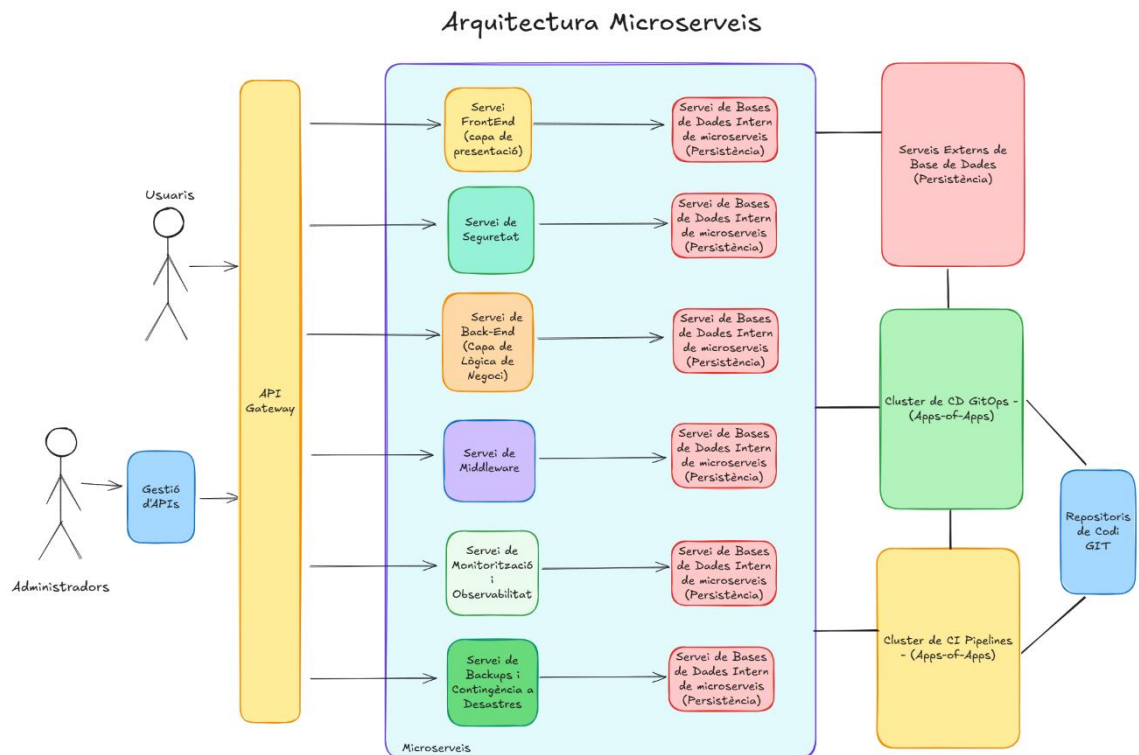


Figura 17 bis. Diagrama alt nivell de components funcionals en una Arquitectura de Microserveis amb la gestió de CI/CD. Creació pròpia

Finalment però fins i tot el més important, identifiquem **el component de seguretat és essencial** per garantir la integritat, confidencialitat i disponibilitat dels sistemes en una arquitectura moderna basada en microserveis. Aquest component actua com un conjunt integrat de mecanismes que protegeixen tant les comunicacions entre serveis com les dades que gestionen.

Les comunicacions internes entre microserveis estan protegides amb **protocols de xifrat com TLS**, que eviten la intercepció de dades durant el seu trànsit. Per gestionar l'accés als recursos del sistema, s'implementen solucions robustes **d'autenticació i autorització** que assegurin que només els usuaris o processos autoritzats puguin interactuar amb els serveis.

A més, el component de seguretat inclou eines específiques per a la **gestió de secrets**, garantint que credencials sensibles, com claus d'accés o tokens, estiguin protegides contra filtracions o accessos indeguts. Al següent apartat explicarem en detall les eines triades quant a seguretat útils en una arquitectura de microserveis.

3.2 Estratègia de Seguretat i Autenticació

La seguretat és un aspecte clau en qualsevol arquitectura de sistemes, especialment en un entorn basat en microserveis, on la fragmentació del sistema implica una major exposició a vulnerabilitats. La transició des d'un monòlit cap a

una arquitectura distribuïda introdueix nous reptes i requereix mesures específiques per protegir tant la infraestructura com les dades.

En una arquitectura de microserveis, les comunicacions internes entre els diferents components són constants i vitals per al funcionament del sistema. Per garantir que aquestes interaccions siguin segures, s'utilitzen mecanismes com el xifrat TLS (Transport Layer Security), que assegura que la informació que circula entre serveis no pugui ser interceptada ni manipulada. A més, sovint es despleguen passarel·les d'API (API Gateways) que no només gestionen les sol·licituds externes, sinó que també actuen com a punts de control per validar l'autenticitat i la integritat de les dades.

Amb molts serveis treballant de manera independent, és fonamental establir un mecanisme centralitzat d'autenticació i autorització. Sistemes com OAuth2 i OpenID Connect permeten gestionar l'accés a recursos de manera consistent i segura, assegurant que cada sol·licitud prové d'un usuari o sistema autoritzat. Aquest enfocament ajuda a prevenir accés no autoritzat i possibles atacs de suplantació.

Les credencials, claus d'accés i altres secrets necessaris per al funcionament del sistema han d'estar protegits adequadament. Eines com **HashiCorp Vault** [\[13\]](#) o **AWS Secrets Manager** permeten gestionar aquests elements de manera segura, garantint que només els serveis o usuaris autoritzats tinguin accés a la informació sensible.

En un sistema distribuït, la detecció d'activitats sospitoses és més complexa però essencial. La supervisió constant dels logs i mètriques a través d'eines d'observabilitat permet identificar patrons inusuals que podrien indicar un intent d'atac. Això es complementa amb tallafocs d'aplicació web (WAF) i sistemes de detecció i prevenció d'intrusions (IDS/IPS), que analitzen el tràfic per identificar i bloquejar amenaces abans que afectin el sistema.

Un aspecte especialment crític és assegurar-se que el sistema compleix amb les normatives de protecció de dades, com el RGPD. Això implica no només protegir les dades contra accessos no autoritzats, sinó també establir polítiques clares sobre com s'emmagatzemen, processen i eliminen les dades personals. Això s'aconsegueix mitjançant controls d'auditoria, xifrat de dades en repòs i en trànsit, i la segmentació adequada de bases de dades per minimitzar l'exposició.

Cap sistema és immune als atacs, per la qual cosa és essencial tenir un pla sòlid de resposta a incidents. Aquest pla inclou processos definits per identificar, contenir i mitigar les conseqüències d'un incident de seguretat, així com per recuperar-se ràpidament per minimitzar l'impacte en els usuaris i el negoci.

Amb aquestes mesures, no només es garanteix la seguretat de l'arquitectura, sinó que també es genera confiança entre els usuaris i es protegeix el valor del sistema a llarg termini.

3.3 Estratègia de Monitorització i Registre

L'estratègia de monitorització i registre és fonamental per mantenir el rendiment, la disponibilitat i la seguretat d'un sistema basat en microserveis. Aquesta estratègia es dissenya per proporcionar visibilitat completa sobre l'estat dels components del sistema, identificar possibles problemes de manera proactiva i garantir la traçabilitat en cas d'incidents.[\[14\]](#)

La **monitorització** implica l'ús d'eines per recopilar dades sobre l'estat i el comportament del sistema en temps real. Això inclou mètriques de rendiment, com ara temps de resposta, ús de CPU, memòria i trànsit de xarxa, així com mètriques específiques dels microserveis, com el volum de transaccions processades o la latència de les comunicacions internes.

S'utilitzen plataformes com Prometheus o Datadog per configurar alertes automàtiques que avisen en cas de detectar anomalies, com un augment sobtat en els errors de resposta o una caiguda de rendiment. Aquesta capacitat permet als equips actuar ràpidament per solucionar problemes abans que afectin els usuaris finals.

L'**observabilitat** complementa la monitorització, proporcionant una visió més profunda de com es comporten els sistemes. Inclou el rastreig distribuït, que permet seguir el flux de sol·licituds a través de múltiples microserveis, així com la correlació de logs, mètriques i traçabilitat per comprendre l'origen dels problemes. Eines com Jaeger o OpenTelemetry són clau per implementar l'observabilitat en un entorn distribuït.

Els **registres** (logs) són una font essencial d'informació per analitzar el comportament del sistema i identificar l'origen dels errors. Els microserveis generen registres detallats que inclouen informació sobre les operacions realitzades, els errors trobats i altres esdeveniments rellevants. Per gestionar grans volums de dades de registre, es poden utilitzar solucions com Elasticsearch o Loki, que permeten indexar, cercar i analitzar aquestes dades de manera eficient.

En conjunt, la monitorització, l'observabilitat i el registre permeten implementar una **estratègia integral** que assegura la disponibilitat del sistema i millora la resposta davant incidents. Això ajuda a garantir que, fins i tot en un entorn complex de microserveis, es pugui mantenir un control ferm sobre el seu rendiment i estabilitat.

4. Planificació Temporal i Recursos

4.1 Desglossament de Fases del Projecte

El projecte es descompon en diverses fases per garantir un seguiment ordenat i metòdic. Aquest desglossament permet estructurar les activitats principals en blocs de treball que es poden gestionar i supervisar amb precisió.

Fase 1: Planificació

La fase de planificació és una etapa fonamental en el desenvolupament d'un projecte. És el moment en què es defineixen les bases per a l'èxit del treball, s'identifiquen els objectius, els recursos necessaris, els terminis i les possibles dificultats. Aquest procés inclou diverses activitats que assegurin que totes les parts interessades estiguin alineades i que el projecte disposi d'un marc clar per avançar.

Objectius

- **Definir els requeriments:** Es detallen les necessitats i expectatives del projecte, ja sigui en termes tècnics, funcionals o de qualitat.
- **Establir un pla temporal:** Es dissenya un cronograma detallat que inclou les fites i terminis per cada fase del projecte.
- **Identificar els recursos:** Es quantifiquen i organitzen els recursos humans, materials i financers necessaris.
- **Mitigar riscos:** Es realitza una anàlisi dels possibles riscos que poden sorgir, juntament amb les mesures preventives o correctives.

Activitats clau

1. **Reunions inicials amb les parts interessades**
 - a. Objectiu: Clarificar els requeriments i establir les prioritats del projecte.
 - b. Resultat: Llista inicial d'objectius i limitacions.
2. **Identificació de riscos**
 - a. Es detecten àrees de vulnerabilitat (com ara problemes tècnics o limitacions de temps).
 - b. Es desenvolupen estratègies de mitigació, com l'assignació de recursos addicionals o plans de contingència.
3. **Creació del cronograma**
 - a. Es desglossen les activitats en tasques més petites, amb terminis específics.
 - b. Es defineixen les dependències entre tasques per optimitzar el flux de treball.
4. **Assignació de recursos**
 - a. Es distribueixen els membres de l'equip segons les seves capacitats i experiència.
 - b. Es planifiquen inversions en infraestructura, eines i formació.

Resultats esperats

Al final de la fase de planificació, el projecte hauria de comptar amb:

- Un pla de treball clar, estructurat en fases i activitats.
- Un cronograma amb terminis definits.
- Una assignació eficient dels recursos disponibles.
- Una estratègia de gestió de riscos.

Aquest conjunt de directrius proporciona la base per avançar amb seguretat cap a les següents fases, reduint la incertesa i millorant l'eficiència en l'execució del projecte.

Fase 2: Disseny del Sistema

La fase de disseny del sistema és essencial per establir les bases arquitectòniques, tècniques i funcionals que regiran la implementació del projecte. Aquesta etapa implica analitzar detalladament els requisits recollits durant la fase de planificació i traduir-los en components, serveis i interaccions clares, assegurant que el sistema compleixi amb les necessitats especificades.

Objectius

- Definir l'arquitectura del sistema de manera que sigui escalable, modular i segura.
- Identificar i dissenyar els components principals del sistema.
- Especificar les interaccions i fluxos de dades entre els components.
- Garantir que el disseny s'alinea amb els objectius del projecte i amb els principis de domini del negoci.

Activitats clau

- 1. Identificació dels components del sistema**
 - a. Es determinen els principals blocs que conformaran el sistema: Front-End, Back-End, Base de Dades, Middleware, Monitorització, i altres.
 - b. Es consideren aspectes com la persistència de dades, la gestió de la comunicació i la seguretat.
- 2. Disseny d'arquitectura**
 - a. Es defineix si l'arquitectura serà monolítica, basada en microserveis o una combinació.
 - b. Es documenten les interaccions entre els components i les seves dependències, incloent-hi protocols de comunicació.
- 3. Planificació de la infraestructura**
 - a. Es detallen les necessitats tècniques, com ara servidors, sistemes d'emmagatzematge, i eines d'automatització.
 - b. Es trien eines per a la gestió del codi (com Git) i orquestració (Kubernetes).
- 4. Documentació i validació del disseny**
 - a. Es crea una documentació tècnica que inclou diagrames d'arquitectura, especificacions de components i esquemes de bases de dades.
 - b. Es comparteix el disseny amb els equips implicats per rebre feedback i ajustar possibles inconsistències.

Resultats esperats

- Un document complet que descrigui l'arquitectura del sistema i els components identificats.
- Diagrames d'interacció i fluxos de treball que descriguin com funcionaran

- conjuntament els components.
- Especificacions tècniques detallades per a la implementació del sistema.
- Assegurament que el sistema compleix amb els requeriments funcionals, normatius i de seguretat.

El disseny del sistema actua com una guia que assegura una transició fluida cap a la fase de desenvolupament, minimitzant ambigüitats i riscos associats amb la implementació.

Fase 3: Implementació

La fase d'implementació és el nucli del projecte, on el disseny definit anteriorment es materialitza en un sistema funcional. Aquesta etapa implica la creació, configuració i integració dels components del sistema, assegurant la seva coherència i eficiència.

Objectius

- Construir i desplegar els components del sistema d'acord amb l'arquitectura dissenyada.
- Garantir que cada component funcioni de manera independent i coordinada amb els altres.
- Realitzar una integració progressiva per validar l'eficàcia del sistema en conjunt.

Activitats clau

- 1. Desenvolupament del codi**
 - a. Implementació dels components de Front-End i Back-End, utilitzant les eines i llenguatges especificats (React, Python, Node.js, etc.).
 - b. Configuració de la base de dades amb les estructures definides durant la fase de disseny.
- 2. Desplegament de la infraestructura**
 - a. Configuració dels entorns de desenvolupament, proves i producció.
 - b. Ús d'eines d'orquestració com Kubernetes per gestionar els microserveis.
 - c. Instal·lació i configuració de middleware per facilitar la comunicació entre components.
- 3. Integració de components**
 - a. Proves inicials d'integració per assegurar que els diferents components poden comunicar-se correctament.
 - b. Resolució de problemes que emergeixin durant aquesta integració.
- 4. Configuració de monitorització i observabilitat**
 - a. Implementació d'eines per supervisar el rendiment del sistema (Prometheus, Grafana, etc.).
 - b. Configuració de registres detallats per capturar incidències i analitzar-les.
- 5. Proves tècniques**
 - a. Execució de proves unitàries, de rendiment i de seguretat per verificar la qualitat del sistema.
 - b. Ajustaments i optimitzacions basats en els resultats de les proves.

Resultats esperats

- Components funcionals desplegats i operatius en els entorns definits.
- Una integració estable entre els diferents serveis i subsistemes.
- Sistemes de monitorització i registre configurats per garantir un seguiment continu del rendiment.
- Un sistema preliminar preparat per a la validació final en la fase de proves.

Consideracions

- **Coordinació contínua:** És crucial mantenir una comunicació fluida entre els equips per evitar duplicitats o inconsistències.
- **Adaptabilitat:** Aquesta fase pot requerir ajustaments ràpids en resposta a problemes inesperats o canvis en els requisits.
- **Documentació:** És important documentar tot el procés per facilitar el manteniment futur i la transferència de coneixement.

La implementació és una fase crítica que converteix les idees en realitat, marcant el pas definitiu cap a un sistema que comenci a prendre forma tangible.

Fase 4: Proves i Optimització

La quarta fase del projecte se centra en assegurar que el sistema desenvolupat compleix els requisits establerts, és estable, eficient i segur. A més, es fan ajustaments finals per garantir una experiència d'usuari òptima i un rendiment adequat abans del desplegament en entorn de producció.

Objectius

- Validar que el sistema satisfà els requeriments tècnics i funcionals establerts.
- Identificar i corregir qualsevol incidència o punt dèbil.
- Optimitzar el rendiment i assegurar la compatibilitat amb les especificacions inicials.

Activitats clau

1. Execució de proves

- a. **Proves unitàries:** Verificar que els components individuals funcionen segons el previst.
- b. **Proves d'integració:** Garantir que els diferents components interactuen correctament.
- c. **Proves de rendiment:** Mesurar la capacitat del sistema sota càrregues elevades per assegurar-ne l'escalabilitat.
- d. **Proves de seguretat:** Detectar i corregir vulnerabilitats potencials, com la protecció de dades o la resistència a ciberatacs.

2. Validació normativa

- a. Revisió de conformitat amb les regulacions aplicables, com el RGPD, per garantir la privadesa i protecció de dades.
 - b. Avaluació de la compatibilitat amb les especificacions tècniques i legals definides.
- 3. Optimització del sistema**
- a. Refinament del codi per millorar la velocitat i l'eficiència dels processos.
 - b. Ajustos finals a la configuració del sistema per aconseguir el millor rendiment possible.
 - c. Implementació de millores basades en els resultats de les proves.
- 4. Simulacions i escenaris reals**
- a. Simulació de situacions reals d'ús per observar el comportament del sistema en entorns semblants als de producció.
 - b. Ajustos per assegurar una transició fluida entre l'entorn de proves i el de producció.
- 5. Documentació de resultats**
- a. Elaboració d'informes detallats sobre el rendiment, incidències detectades i mesures correctives aplicades.
 - b. Preparació de guies i manuals per a l'usuari final i l'equip tècnic.

Resultats esperats

- Un sistema totalment funcional i preparat per al desplegament.
- Resolució de totes les incidències detectades durant les proves.
- Documentació completa i actualitzada que detalli el rendiment, canvis i ajustaments realitzats.

Consideracions

- **Temps addicional per a correccions:** És important tenir una certa flexibilitat per solucionar problemes imprevistos.
- **Validació contínua:** Mantenir una retroalimentació constant amb els equips implicats per assegurar que els objectius s'estan assolint.
- **Documentació exhaustiva:** Una bona documentació facilita el manteniment futur i assegura la transferència de coneixement.

La fase de proves i optimització és fonamental per consolidar el treball realitzat en les fases anteriors i garantir que el sistema no només compleix amb les expectatives sinó que supera possibles desafiaments en l'entorn de producció.

Fase 5: Lliurament i Documentació

La cinquena i última fase del projecte se centra en preparar el lliurament formal del sistema complet i assegurar-se que tota la documentació associada està actualitzada i correcta. Aquesta etapa marca el tancament oficial del projecte i garanteix que tant els usuaris finals com els equips tècnics disposin de les eines i coneixements necessaris per operar i mantenir el sistema.

Objectius

- Formalitzar el lliurament del sistema complet als stakeholders.
- Proporcionar tota la documentació tècnica i d'usuari necessària.
- Assegurar la transició sense problemes cap a l'entorn de producció o ús.

Activitats clau

1. Revisió final del sistema

- a. Verificació que tots els components compleixen els requisits i els estàndards establerts.
- b. Realització d'una última sèrie de proves per confirmar l'estabilitat i el rendiment del sistema.

2. Preparació de la documentació

- a. **Documentació tècnica:** Inclou especificacions, arquitectura, processos d'instal·lació, configuració i manteniment.
- b. **Manuais d'usuari:** Guies detallades orientades a facilitar l'ús del sistema per als usuaris finals.
- c. **Registres de proves:** Informes sobre els resultats de les proves realitzades, incloent incidències resoltes i ajustaments aplicats.

3. Formació i suport inicial

- a. Sessions de formació per als usuaris i els equips tècnics responsables de la gestió i manteniment.
- b. Assignació d'un període de suport post-implementació per resoldre dubtes o problemes inicials.

4. Lliurament formal

- a. Transferència oficial del sistema als stakeholders, incloent les credencials d'accés, les configuracions finals i la documentació.
- b. Sessió de presentació del sistema, destacant les funcionalitats clau i els beneficis principals.

5. Valoració final i tancament del projecte

- a. Reunió amb els stakeholders per revisar l'estat final del projecte i recollir feedback.
- b. Elaboració d'un informe final que reculli tot el cicle del projecte, els resultats obtinguts i recomanacions per a futures millores.

Resultats esperats

- Sistema completament funcional, lliurat i operatiu segons les especificacions.
- Usuaris formats i preparats per utilitzar el sistema.
- Documentació completa i detallada, que faciliti el manteniment futur i l'escalabilitat del sistema.
- Feedback dels stakeholders que serveixi per tancar el projecte i obtenir aprenentatges de cara a iniciatives futures.

Consideracions

- **Claredat i accessibilitat:** La documentació ha de ser fàcil de comprendre tant per a tècnics com per a usuaris no tècnics.
- **Període de transició:** És recomanable mantenir un període de suport inicial per minimitzar problemes.
- **Valoració post-implementació:** Les opinions dels usuaris i responsables són claus per mesurar l'èxit del projecte.

La fase de lliurament i documentació és fonamental per assegurar que el projecte finalitza de manera ordenada, amb tots els elements necessaris per garantir la continuïtat i eficiència del sistema implementat.

4.2 Estratègia d'Integració

La **estratègia d'integració** basada en Helm Charts amb ArgoCD afegeix un enfocament GitOps per gestionar, desplegar i versionar aplicacions de manera declarativa en entorns Kubernetes. Aquesta metodologia ofereix una integració contínua i una entrega contínua (CI/CD) robusta i eficient, especialment en sistemes basats en microserveis.

Ús de Helm Charts

Què és Helm i per què és important?[\[15\]](#)

Helm és una eina essencial en l'ecosistema Kubernetes perquè facilita la gestió d'aplicacions mitjançant un sistema de paquets coneguts com **charts**. Un Helm Chart encapsula tots els manifestos YAML necessaris per desplegar i configurar una aplicació, incloent-hi serveis, secrets, ingressos, i altres recursos. Aquesta funcionalitat redueix significativament la complexitat de treballar directament amb YAML, especialment quan les aplicacions tenen múltiples components o depenen de serveis externs.

En lloc de gestionar manualment les configuracions i mantenir una multitud de fitxers YAML per als diferents entorns (desenvolupament, proves, producció), Helm permet empaquetar aquestes configuracions en un format reutilitzable i versionable. Aquesta reutilització fa que les aplicacions siguin fàcils de desplegar i mantenir en qualsevol entorn Kubernetes.

Per què Helm és útil en l'estratègia d'integració?

Quan es tracta d'automatitzar desplegaments i garantir consistència en diferents etapes de desenvolupament, Helm és clau. Per exemple:

- Si una aplicació necessita un conjunt complex de serveis que depenen l'un de l'altre, Helm permet definir-ho tot dins d'un sol Chart. Això inclou bases de dades, serveis backend, i altres recursos necessaris.

- Les versions de Helm Charts garanteixen que, encara que es produeixi un error en un desplegament, es pugui fer un **rollback** a una versió anterior sense complicacions.

Per què necessitem Helm?

- 1. Gestió simplificada:**
 - a. Els manifestos YAML necessaris per desplegar aplicacions en Kubernetes poden ser extensos i difícils de gestionar. Helm facilita aquest procés empaquetant aquests manifestos en un únic chart.
- 2. Versionat i reutilització:**
 - a. Amb Helm, les aplicacions poden tenir diferents versions i configuracions associades a entorns específics (desenvolupament, producció, etc.).
- 3. Desplegaments repetibles:**
 - a. Els Helm Charts garanteixen que les aplicacions es despleguin de manera consistent, independentment de l'entorn.
- 4. Facilitat de compartició:**
 - a. Els charts es poden emmagatzemar en repositoris públics o privats, permetent compartir configuracions amb equips o comunitats.

Funcionalitats clau de Helm

- 1. Charts:**
 - a. Paquets que contenen tots els manifests YAML necessaris per desplegar una aplicació o servei.
- 2. Releases:**
 - a. Cada vegada que es desplega un Helm Chart, es crea una release, que representa una instància específica del chart.
- 3. Rollbacks:**
 - a. Permet tornar a versions anteriors si hi ha errors durant o després del desplegament.

La sinergia amb ArgoCD

ArgoCD, un operador GitOps per Kubernetes, està perfectament integrat amb Helm. Això permet a ArgoCD desplegar aplicacions basades en Helm Charts directament des d'un repositori Git. Aquesta combinació potencia l'automatització i la supervisió, assegurant que l'estat definit en codi al repositori es correspongui amb el que està desplegat en el clúster.

Amb aquesta integració, ArgoCD pot:

- Detectar canvis en els fitxers Helm del repositori Git.

- Sincronitzar aquests canvis automàticament amb el clúster Kubernetes.
- Proporcionar un historial detallat de desplegaments i canvis, simplificant les auditories.

Per exemple, si tens una aplicació composta per un backend, un frontend i una base de dades, pots empaquetar cada component dins del seu Helm Chart i definir configuracions específiques per a cada entorn. ArgoCD s'encarregarà d'orquestrar els desplegaments segons els canvis en el repositori.

L'impacte d'aquest enfocament en el cicle CI/CD

Aquest sistema també transforma les pipelines de CI/CD. Desplegar i integrar components mitjançant Helm Charts redueix errors humans i permet un desplegament coherent i repetible. Quan es combinen Helm i ArgoCD, qualsevol canvi al codi font (o als manifestos Helm) pot iniciar automàticament un cicle de desplegament al clúster.

En un escenari pràctic es pot:

1. Configurar una pipeline en GitLab que generi els artefactes de l'aplicació i actualitzi els Helm Charts al repositori Git.
2. Utilitzar ArgoCD per sincronitzar el clúster Kubernetes amb aquests charts actualitzats, aplicant automàticament els canvis al teu entorn.

Aquest enfocament no només simplifica el procés d'integració, sinó que també millora la traçabilitat i escalabilitat del sistema.

ArgoCD per a la Integració

1. **Automatització declarativa amb GitOps:**
 - a. ArgoCD supervisa un repositori Git que conté els manifests de Kubernetes o els Helm Charts i assegura que l'estat del clúster coincideixi amb l'estat declarat al repositori.
 - b. Si es detecten desalineacions, ArgoCD sincronitza automàticament o manualment el clúster per corregir-les.
2. **App of Apps Pattern:**
 - a. Mitjançant l'estratègia "App of Apps", ArgoCD pot gestionar múltiples aplicacions des d'un sol manifest, simplificant l'organització i el control de diversos Helm Charts.
 - b. Cada microservei o grup de microserveis es pot gestionar com una aplicació individual, cosa que millora la modularitat i el desacoblament.

Flux de treball típic amb Helm Charts i ArgoCD

1. **Desenvolupament i manteniment del Helm Chart:**
 - a. Els desenvolupadors creen o actualitzen Helm Charts i els emmagatzemen en un repositori Git.
 - b. Els valors de configuració (per exemple, per a entorns com desenvolupament, preproducció o producció) es defineixen en fitxers

values.yaml.

2. Desplegament automatitzat:

- a. ArgoCD supervisa el repositori Git i detecta canvis en els charts o els manifests associats.
- b. Un cop detectat un canvi, ArgoCD aplica les actualitzacions al clúster Kubernetes.

3. Integració amb pipelines CI/CD:

- a. GitLab CI/CD o altres eines poden executar proves i validar els canvis als Helm Charts abans de fer un merge al repositori principal.
- b. Això garanteix que només els canvis aprovats siguin sincronitzats amb el clúster.

Beneficis d'aquesta Estratègia

- **Consistència:** L'ús d'Helm Charts amb ArgoCD manté una descripció declarativa i centralitzada de l'estat del sistema.
- **Automatització:** Redueix els errors manuals gràcies a la sincronització automàtica dels clústers.
- **Escalabilitat:** La combinació d'ArgoCD i Helm facilita la gestió d'aplicacions complexes amb múltiples microserveis.
- **Traçabilitat:** L'estat del sistema queda registrat al repositori Git, proporcionant una traçabilitat completa dels canvis.

Exemple pràctic

Un exemple pràctic d'utilitzar Helm i ArgoCD per migrar una aplicació monolítica que corre en un servidor LAMP (Linux, Apache, MySQL, PHP) cap a una arquitectura basada en Kubernetes pot ser descrit com segueix:

Escenari inicial

La nostra aplicació està composta per:

1. Un servidor Apache que serveix l'aplicació web.
2. Un backend escrit en PHP.
3. Una base de dades MySQL.
4. Tot això s'executa en un servidor físic o màquina virtual única.

Objectiu de la migració

Dividir l'aplicació en components independents que puguin ser gestionats i escalats de manera autònoma dins d'un clúster Kubernetes. Això també permet aplicar pràctiques modernes com la CI/CD i GitOps.

Migració a Kubernetes amb Helm i ArgoCD

Divisió del Monòlit

- Identifica els components principals de l'aplicació:
 - Frontend: PHP i Apache.
 - Backend: Business logic en PHP.
 - Base de dades: MySQL.

Creació de Helm Charts per a cada component:

- **Chart del Frontend:**
 - Es defineix un deployment per al servidor Apache amb el codi PHP.
- **Chart del Backend:**
 - Si és necessari, crea un servei separant la lògica del backend.
- **Chart de la Base de Dades:**
 - Utilitza una base de dades MySQL amb persistència de volums.

Creació de Helm Charts per a cada component transversal:

A Kubernetes es gestiona tota la infraestructura mitjançant codi, així necessitarem configurar Charts per la resta de components que donen servei a la nostra aplicació de manera transversal:

- **Chart del Servei:**
 - Defineix serveis per exposar els components dins de Kubernetes.
- **Chart del Ingress:**
 - Configura un enllaç (Ingress) que permet l'accés extern al servei de l'aplicació (FrontEnd)
- **Chart de Escalabilitat:**
 - Configura l'escalabilitat que tindrà l'aplicació de manera automàtica per optimitzar recursos com CPU i Memòria.
- **Chart del Storage:**
 - Les peticions d'espai d'emmagatzematge dinàmic es defineixen aquí on es vinculen aquí.
- **Chart dels Secrets:**
 - Emmagatzema informació sensible com les contrasenyes de MySQL de manera segura.

Preparació del Repositori Git

- Haurem de crear un repositori Git que permeti gestionar cada component de forma independent.
- Haurem d'encloure els manifestos de Helm:
 - **values.yaml** per gestionar configuracions específiques per entorns (dev,

- prod).
- Scripts de deploy compatibles amb ArgoCD.

Deploy inicial

Amb Helm, empaqueta i desplega cada component per separat:

```
helm install frontend ./frontend-chart
helm install backend ./backend-chart
helm install database ./database-chart
```

Això crea pods, serveis i ingressos de manera aïllada.

Automatització amb ArgoCD

- Per la configuració d'ArgoCD per sincronitzar el repositori Git amb el clúster Kubernetes hauria de seguir els passos següents:
 - Defineix una estructura App-of-Apps, on cada component Helm Chart és una aplicació gestionada independentment.
 - Exemple d'un fitxer declaratiu d'ArgoCD:

```
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: frontend
  namespace: argocd
spec:
  source:
    repoURL: 'https://github.com/tu-repositori/frontend-chart'
    targetRevision: HEAD
    path: .
  destination:
    server: 'https://kubernetes.default.svc'
    namespace: default
    project: default
```

- Això permet que qualsevol canvi en el repositori (per exemple, un nou deployment) es reflecteixi automàticament al clúster.

Test i Optimització

- Realització de proves unitàries i d'integració per garantir que cada component funciona de manera independent i conjuntament.
- Monitorització de l'aplicació amb eines com Prometheus i Grafana (configurades com Helm Charts).
- Control sobre l'escalabilitat dels components basant-te en les necessitats:
 - Augmenta els pods del backend si hi ha molta càrrega en les peticions.

Beneficis

1. **Modularitat:** Cada component es gestiona i desplega de manera independent.
2. **Escalabilitat:** Es pot escalar la base de dades, el frontend o el backend segons les necessitats.
3. **Automatització:** ArgoCD assegura que el clúster estigui sempre alineat amb el codi al repositori.
4. **Flexibilitat:** L'ús de Helm permet canvis fàcils en les configuracions per a diferents entorns.

4.3 Pla de Proves i Integració

El pla de proves i integració garanteix que totes les parts del sistema funcionin correctament tant de forma individual com conjunta. Aquest pla és essencial per assegurar la qualitat i la fiabilitat del sistema.

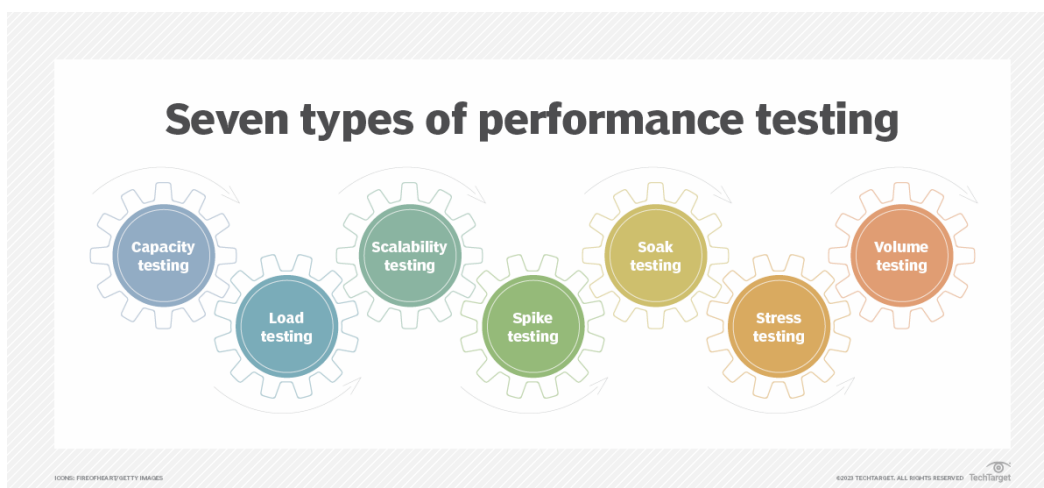


Figura 18. Els set tipus de proves de rendiment d'una aplicació- [techtarget](#)

Proves Unitàries

Les proves unitàries són una tècnica essencial en el desenvolupament de programari que permet verificar el funcionament correcte de components individuals (generalment funcions o mòduls) d'un sistema. Aquest tipus de proves se centren en comprovar que cadascun dels elements aïllats compleixi el seu objectiu funcional segons els requisits establerts.

Objectius de les Proves Unitàries

- **Validar funcionalitats a nivell de component:** Assegurar que cada unitat de codi es comporta segons les especificacions.
- **Detectar errors primerencs:** Identificar defectes en fases inicials del desenvolupament, minimitzant així costos i esforços.
- **Assegurar el comportament previsible:** Garantir que cada unitat respon correctament davant d'entrades vàlides i gestiona adequadament situacions

d'error o excepcions.

- **Facilitar canvis:** Millorar la seguretat a l'hora de fer modificacions, ja que es poden detectar ràpidament efectes col·laterals.

Característiques Principals

1. **Cobertura funcional:** Es dissenyen per cobrir totes les possibles variants de comportament d'una unitat.
2. **Aïllament:** Les proves unitàries s'executen en un entorn controlat on la unitat es prova en solitari, sense dependències d'altres components.
3. **Automatització:** Sovint es desenvolupen conjuntament amb el codi font utilitzant frameworks com JUnit (Java), pytest (Python), Jest (JavaScript), entre d'altres.
4. **Velocitat:** Com són molt específiques i es centren en mòduls petits, la seva execució és ràpida.

Estratègia de Desenvolupament

- **Identificació de casos de prova:** Es defineixen escenaris per verificar el comportament correcte i els errors potencials de la unitat.
- **Creació de mocks i stubs:** En cas que una unitat depengui d'altres components, s'usen objectes simulats (mocks) per reproduir el seu comportament.
- **Execució repetida:** Les proves s'han d'executar cada vegada que es modifica el codi per garantir que no hi hagi regressions.
- **Cobertura de codi:** Es procura que les proves cobreixin la major quantitat possible de línies i branques del codi.

Exemple Pràctic

Imaginem una funció que calcula la suma d'una llista de números. Les proves unitàries per a aquesta funció poden incloure:

1. Verificar que la funció retorna 0 per una llista buida.
2. Comprovar que retorna el valor correcte per una llista amb un sol número.
3. Testar amb diversos números, incloent valors negatius i decimals.
4. Gestionar errors: Passar un valor no vàlid com una cadena o un objecte i verificar que es llenci una excepció.

Beneficis

- **Qualitat del codi:** Promou un codi més robust, ja que els desenvolupadors escriuen pensant en com provar-lo.
- **Documentació implícita:** Les proves sovint reflecteixen com s'espera que funcioni el codi.
- **Facilitat de manteniment:** Quan es fa una modificació, les proves unitàries ajuden a assegurar que el canvi no introdueixi nous errors.

Límits i Desafiaments

- **Temps addicional:** Desenvolupar proves requereix una inversió inicial de temps.
- **Cobertura limitada:** Les proves unitàries només validen el funcionament d'una unitat; poden no detectar problemes que apareixen en la integració entre components.
- **Actualització constant:** Les proves han de ser revisades i actualitzades a mesura que el codi evoluciona.

Proves d'Integració

Les proves d'integració són essencials en sistemes de microserveis per garantir que els diversos components interactuen correctament. En aquest context, s'enfronten reptes específics, com la naturalesa distribuïda del sistema i la interdependència entre serveis. Segons Martin Fowler [17], aquestes proves ajuden a verificar que els serveis poden comunicar-se i comportar-se adequadament quan col·laboren per oferir funcionalitats completes.

Aquestes proves asseguren que, malgrat l'autonomia i independència de cada microservei, el conjunt del sistema es comporta com un tot coherent i funcional. Això és especialment important en sistemes escalables i d'alt rendiment, on la integració deficient pot tenir efectes cascada.

Objectius de les Proves d'Integració

1. **Validació de la Comunicació entre Serveis:** Comprovar que les interfícies i protocols utilitzats (com REST, gRPC o missatgeria asíncrona) són compatibles i funcionen sense errors.
2. **Confirmació de les Contractes d'API:** Assegurar que els serveis compleixen els contractes establerts en termes de dades, esquemes i formats.
3. **Proves de Casos Límits:** Avaluar situacions amb alta latència, pèrdua de missatges o errors de xarxa.
4. **Detecció de Problemes de Configuració:** Verificar que les configuracions compartides o específiques del servei no interfereixen amb la funcionalitat general.

Estratègies per Realitzar Proves d'Integració

- **Mocking i Emulació:** Utilitzar simulacions d'altres serveis per provar un servei específic en aïllament. Aquesta tècnica ajuda a evitar dependències en serveis encara no desenvolupats.
- **Contract Testing:** Provar que els serveis compleixen els contractes predefinits sense necessitat d'una integració completa en l'entorn real. Eines com Pact són útils per implementar aquesta tècnica.
- **End-to-End Testing Seleccionat:** Tot i que no és estrictament una prova d'integració, una selecció de proves end-to-end pot ajudar a validar la

integració de serveis crítics sense executar tests massa amplis.

Reptes Comuns

1. **Gestió de Dades Compartides:** Sovint, els serveis comparteixen accés a bases de dades o recursos comuns. És fonamental provar que aquests recursos no creen colls d'ampolla ni problemes de coherència.
2. **Sincronització entre Serveis:** En sistemes on els serveis actuen de manera asíncrona, cal provar si els esdeveniments i les respostes s'envien i reben en l'ordre esperat.
3. **Impacte del Versionat:** En arquitectures de microserveis, diferents serveis poden tenir versions diferents al mateix temps. És crucial provar que aquests serveis poden coexistir i treballar junts.

Eines Populars

- **Postman o SoapUI:** Per provar les APIs.
- **WireMock:** Per simular serveis dependents.
- **K6:** Per avaluar la latència i el rendiment de les interaccions entre serveis.

Proves Funcionals

Les proves funcionals tenen com a objectiu principal assegurar que el sistema compleix amb els requeriments funcionals establerts. En el context d'un sistema complex com una arquitectura de microserveis, aquestes proves es concentren en verificar que cada funcionalitat treballa correctament des de la perspectiva de l'usuari final, independentment de com s'implementa internament.

Objectius Principals

1. **Validació de Requeriments:** Verificar que totes les funcionalitats especificades pels documents de requeriments es compleixen adequadament.
2. **Perspectiva de l'Usuari Final:** Assegurar que el sistema funciona correctament segons l'experiència esperada pels usuaris.
3. **Detecció d'Errors Operacionals:** Identificar problemes que podrien sorgir durant l'ús diari del sistema.

Característiques de les Proves Funcionals

- **Orientació a la Funcionalitat:** Es centra en què fa el sistema, no en com ho fa.
- **Black-Box Testing:** Els testers no necessiten conèixer la implementació interna. Es basen en inputs i outputs.
- **Automatització Recomanada:** Tot i que algunes proves poden ser manuals, eines com Selenium, TestComplete o Cypress poden facilitar l'execució de

proves recurrents.

Tipus de Proves Funcionals

1. **Proves de Regressió:** Asseguren que canvis o millores en el sistema no introdueixen errors en funcionalitats existents.
2. **Proves de Casos Especials:** Avaluen situacions límit o menys comunes d'ús del sistema.
3. **Validació de Fluxos de Treball Complets:** Confirmin que els fluxos d'operació es comporten correctament i permeten a l'usuari completar les seves tasques.

Estratègia d'Execució

1. **Definir Escenaris d'Ús:** Crear casos de prova que reflecteixin situacions reals d'ús.
2. **Avaluar Variacions d'Entrades:** Provar diferents combinacions d'inputs per assegurar la robustesa del sistema.
3. **Integració amb Proves Automàtiques:** Incorporar eines per executar proves repetidament en diferents entorns.

Exemple Pràctic

En un sistema de microserveis, les proves funcionals podrien verificar, per exemple, que un usuari pot crear, editar i eliminar una comanda en una aplicació d'e-commerce. Això implicaria provar els següents serveis:

- **Servei d'Autenticació:** Per verificar si l'usuari té accés.
- **Servei de Gestió de Comandes:** Per crear o actualitzar les comandes.
- **Servei d'Inventari:** Per assegurar que els productes estan disponibles.
- **Servei de Pagament:** Per validar que les transaccions es processen correctament.

Aquestes proves garanteixen que els diferents serveis col·laboren correctament per oferir una experiència fluida.

Reptes Comuns

- **Complexitat en els Fluxos de Treball:** Quan els serveis depenen molt entre si, les proves poden necessitar dades de múltiples serveis.
- **Interaccions Asíncrones:** La verificació de funcionalitats que depenen d'esdeveniments asíncrons pot ser més complicada.
- **Manteniment de Proves:** Amb l'evolució del sistema, cal actualitzar els casos de prova per reflectir nous requeriments.

Les proves funcionals són crucials per garantir la qualitat i satisfacció de l'usuari final, assegurant que el sistema compleix els requeriments de manera fiable i

consistent.

Proves de Rendiment

Les proves de rendiment són una part fonamental del procés de verificació d'un sistema, especialment en arquitectures distribuïdes com els microserveis. Aquestes proves s'encarreguen de mesurar la capacitat del sistema per respondre sota diferents càrregues de treball, assegurant que pot mantenir un nivell acceptable de rendiment i escalabilitat.

Objectius Principals

1. **Avaluar la Capacitat de Resposta:** Verificar el temps que tarda el sistema a respondre a diferents volums de peticions.
2. **Mesurar l'Escalabilitat:** Analitzar com el sistema es comporta quan es redueix o incrementa la càrrega.
3. **Identificar Colls d'Ampolla:** Localitzar components que limiten el rendiment global.
4. **Provar l'Estabilitat:** Assegurar que el sistema pot mantenir-se operatiu durant llargs períodes de càrrega.

Tipus de Proves de Rendiment

Proves de Càrrega:

Simulen un volum de peticions normal per comprovar si el sistema funciona correctament en condicions previstes.

Exemple: Verificar que un servei d'e-commerce pot gestionar 10.000 comandes simultànies.

Proves d'Estrès:

Es força el sistema més enllà dels seus límits per veure com respon i identificar el punt de fallida.

Exemple: Simular un augment sobtat de peticions durant una campanya de vendes.

Estratègia d'Execució

1. **Definició d'Objectius:** Determinar els límits esperats per a temps de resposta, ús de recursos i càrregues màximes.
2. **Creació d'Escenaris Realistes:** Simular càrregues que reflecteixin l'ús real dels usuaris finals.
3. **Execució amb Eines Especialitzades:** Utilitzar plataformes com JMeter, Gatling o Locust per generar càrrega i mesurar el rendiment.
4. **Anàlisi de Resultats:** Identificar colls d'ampolla, com poden ser una base de

dades saturada, problemes de xarxa o serveis amb temps de resposta elevat.

5. **Iteració:** Proposar solucions per millorar el rendiment i repetir les proves per verificar els ajustos.

Proves de Rendiment en Microserveis

En una arquitectura de microserveis, el rendiment pot variar significativament depenent de com es comuniquen els components:

- **Temps de Resposta entre Serveis:** Els serveis han de ser capaços de processar peticions ràpidament i sense retards.
- **Latència en la Xarxa:** Es mesura la latència introduïda per la comunicació entre serveis.
- **Equilibri de Càrrega:** Assegurar que els serveis es poden distribuir correctament entre instàncies per evitar sobrecàrregues.
- **Dependències Externes:** Analitzar com es comporten els microserveis amb bases de dades, API de tercers o altres dependències.

Exemple Pràctic

En un sistema de comerç electrònic, les proves de rendiment podrien incloure:

1. Simular milers de clients navegant pel catàleg de productes.
2. Provar la capacitat de gestionar centenars de comandes per segon.
3. Mesurar l'impacte d'un augment en la consulta de la base de dades durant períodes promocionals.

Reptes Comuns

- **Entorns Heterogenis:** Els resultats poden variar entre entorns de prova i producció.
- **Cost de les Proves:** Generar càrregues elevades pot requerir infraestructura addicional.
- **Simulació Realista:** És difícil replicar de manera precisa totes les condicions del món real.

Aquestes proves són essencials per garantir que el sistema pugui complir amb les expectatives d'ús, especialment durant períodes de gran demanda o escenaris crítics.

Proves de Resistència i d'Escalabilitat

Les **proves de resistència** i les **proves d'escalabilitat** són components essencials per garantir que un sistema informàtic funcioni de manera eficaç i eficient en condicions extremes i sota càrregues creixents. Tot i que ambdues tenen objectius similars, també presenten diferències substancials en els aspectes que es

mesuren.

Proves de Resistència

Les **proves de resistència**, també conegudes com **soak testing** [18], avaluen com el sistema es comporta quan s'exposa a una càrrega elevada durant un període llarg de temps. L'objectiu és identificar qualsevol degradació del rendiment que es pugui produir a mesura que el sistema opera sota una càrrega sostinguda. Aquest tipus de proves ajuda a identificar problemes que podrien no ser evidents durant les proves de càrrega a curt termini.

Objectius:

- Comprovar que el sistema no presenta fuites de recursos (per exemple, memòria).
- Identificar qualsevol degradació de rendiment (com una disminució de la velocitat o un augment en el temps de resposta).
- Validar la robustesa i la fiabilitat a llarg termini.

Exemple: En el cas d'un servidor web, les proves de resistència poden implicar l'execució de peticions durant diversos dies per observar si el rendiment es manté constant o si comencen a aparèixer errors després de llargues hores d'ús continuat.

Proves d'Escalabilitat

Les **proves d'escalabilitat** se centren en la capacitat del sistema per augmentar el seu rendiment en resposta a un augment de la càrrega. Aquestes proves són fonamentals per garantir que el sistema pugui créixer per atendre més usuaris o més trànsit sense perdre qualitat o eficiència. L'objectiu d'aquestes proves és identificar com un sistema pot suportar el creixement de la seva infraestructura i el trànsit a mesura que es modifiquen les condicions.

Objectius:

- Comprovar la capacitat del sistema per adaptar-se a l'augment de la càrrega, ja sigui per augmentar el nombre d'usuaris o les demandes d'operacions.
- Validar que el sistema segueixi rendint de manera òptima a mesura que s'afegeixen més recursos.
- Comprovar que el sistema pot distribuir la càrrega entre diferents components de manera eficaç per evitar colls d'ampolla.

Exemple: Si una plataforma de comerç electrònic preveu una gran quantitat de visites durant les festes, les proves d'escalabilitat poden simular una gran quantitat d'usuaris per verificar si els servidors i la base de dades poden gestionar aquesta càrrega afegida sense experimentar baixades en el rendiment o temps de resposta excessius.

Diferències Clau Entre Proves de Resistència i Escalabilitat

1. **Durada vs. Càrrega:** Les proves de resistència es realitzen a llarg termini per avaluar la resistència d'un sistema sota càrregues sostingudes, mentre que les proves d'escalabilitat busquen determinar com el sistema gestiona les càrregues creixents o de gran envergadura a curt termini.
2. **Focalització en els Recursos:** Les proves de resistència busquen identificar problemes de consum de recursos (com fuites de memòria), mentre que les proves d'escalabilitat s'enfoquen en l'adaptabilitat del sistema a més càrrega i la seva capacitat per distribuir la càrrega de manera eficient.
3. **Objectius:** Les proves de resistència volen confirmar que el sistema no experimenta falles de rendiment amb el temps, mentre que les proves d'escalabilitat volen assegurar-se que el sistema pot augmentar els seus recursos de manera eficient per mantenir la qualitat del servei.

Combinació d'Ambdues Proves

Per garantir la fiabilitat global d'un sistema en producció, és essencial realitzar tant proves de resistència com d'escalabilitat. D'aquesta manera, podem estar segurs que el sistema no només aguantarà bé en escenaris normals i estressats, sinó que també podrà créixer adequadament davant una demanda creixent.

Proves de Seguretat

Les **proves de seguretat** són fonamentals per garantir que els sistemes informàtics siguin resistents davant possibles atacs, vulnerabilitats i altres riscos de seguretat. Aquestes proves s'orienten a identificar, mitigar i corregir les amenaces que podrien comprometre la integritat, la confidencialitat i la disponibilitat de les dades i serveis.

Proves de penetració (Penetration Testing)

Aquest tipus de prova simula atacs realistes contra el sistema per identificar vulnerabilitats en els sistemes de seguretat, com ara les errades de configuració, les vulnerabilitats de programari i les contrasenyes febles. S'intenta explotar les debilitats per veure si un atacant podria accedir a informació confidencial o controlar el sistema. [\[19\]](#)

Avaluació de vulnerabilitats

Aquestes proves es focalitzen en identificar i catalogar possibles vulnerabilitats dins d'un sistema. Mitjançant eines automàtiques o manuals, els equips de seguretat escanejen l'aplicació, la xarxa i el sistema per detectar punts febles que puguin ser explotats per atacants amb eines com ZAP [\[20\]](#).

Revisió de codi segur

Aquesta prova implica revisar el codi font d'una aplicació per identificar defectes de seguretat, com per exemple injeccions SQL, errors de configuració de seguretat o ús incorrecte de biblioteques de tercers. Aquesta revisió pot ser feta de manera manual o mitjançant eines automatitzades.

Proves de resistència a denegacions de servei (DoS/DDoS)

Les proves de resistència analitzen com el sistema respon davant d'atacs de denegació de servei, on un atacant intenta sobrecarregar el sistema amb un volum excessiu de sol·licituds. Aquestes proves ajuden a validar si el sistema pot suportar una alta càrrega sense caure o perdre serveis.

Proves d'autenticació i autorització

Aquestes proves verifiquen que el sistema implementi correctament les polítiques d'autenticació (com el login de l'usuari) i d'autorització (control d'accés a funcions o recursos). Es comprova que no es permeti l'accés no autoritzat, que les contrasenyes siguin segures i que les sessions es gestionin de manera correcta.

Importància de les Proves de Seguretat

Les proves de seguretat ajuden a prevenir possibles vulnerabilitats abans que siguin explotades per atacants maliciosos. Això és crucial en un entorn de microserveis o aplicacions basades en el núvol, on la fragmentació dels components pot augmentar la superfície d'atac i fer més difícil la detecció de riscos.

Organismes com el NIST manté la National Vulnerability Database (NVD)[\[21\]](#), un dipòsit d'informació sobre defectes de programari i maquinari que poden comprometre la seguretat informàtica. Aquesta és una peça clau de la infraestructura de ciberseguretat. Hi ha una acumulació creixent de vulnerabilitats enviades a l'NVD i que requereixen anàlisi. A cada vulnerabilitat se li assigna un identificador de CVE (Common Vulnerabilities and Exposures), una descripció, dates i comentaris únics, cosa que facilita el seguiment i la gestió de les vulnerabilitats a diferents plataformes.

Les proves de seguretat han de ser un procés continuat, en què el sistema es revisa regularment a mesura que evoluciona. Aquestes proves no només detecten riscos actuals, sinó que també ajuden a establir millores en el codi i en la infraestructura per mantenir una postura de seguretat robusta a llarg termini.

5. Avaluació i Gestió de Riscos

5.1 Identificació de Biaixos i Mesures Pal·liatives

Optimització dels Helm Charts per Reutilització amb ArgoCD

Per millorar l'eficiència i la reutilització dels Helm Charts al desplegar components de frontend, backend, bases de dades o altres microserveis, es pot adoptar una estratègia que inclogui l'ús de plantilles i configuracions globals. Aquesta metodologia permet gestionar múltiples entorns i variacions de producte mitjançant una configuració modular i flexible.

Estructura dels Helm Charts Reutilitzables

Plantilles comunes:

Cada component (frontend, backend, base de dades) comparteix plantilles bàsiques per a recursos com:

1. *Deployments*
2. *Services*
3. Configuracions de *Ingress*
4. *Secrets* i *ConfigMaps*

Exemple: Una plantilla genèrica per a un *Deployment* podria incloure etiquetes i anotacions configurables segons l'entorn.

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: {{ .Values.app.name }}
  labels:
    app: {{ .Values.app.name }}
    environment: {{ .Values.environment }}
spec:
  replicas: {{ .Values.replicaCount }}
  template:
    metadata:
      labels:
        app: {{ .Values.app.name }}
        environment: {{ .Values.environment }}
    spec:
      containers:
        - name: {{ .Values.app.name }}
          image: {{ .Values.image.repository }}:{{ .Values.image.tag }}
```

Fitxers de configuració globals:

Es defineixen en un fitxer comú, per exemple, `values.yaml`, que conté totes les variables ajustables segons l'entorn o producte.

Exemple:

```
app:
  name: product-backend
image:
  repository: myregistry/product-backend
  tag: v1.2.3
environment: staging
replicaCount: 2
```

Parametrització per entorns:

Els entorns (com `staging`, `production`, etc.) poden tenir arxius de configuració separats (`values-staging.yaml`, `values-production.yaml`) o claus específiques al fitxer global. Això permet ajustar rèpliques, imatges, recursos i secrets per a cada entorn sense duplicar Helm Charts.

Exemple per `staging`:

```
environment: staging
replicaCount: 1
image:
  tag: v1.2.3-staging
```

Gestió amb ArgoCD

ArgoCD permet integrar aquesta estratègia mitjançant:

1. App of Apps:

Utilitzar un enfocament "App of Apps" per gestionar múltiples aplicacions o components des d'un sol Helm Chart principal.

Exemple:

```
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: app-of-apps
spec:
  destination:
    server: https://kubernetes.default.svc
    namespace: argocd
  source:
    repoURL: 'https://github.com/my-org/helm-charts'
    path: 'apps'
    targetRevision: HEAD
  project: default
```

2. ConfigMaps o Secrets compartits:

Per gestionar les variables globals que canvien segons l'entorn, com ara URLs, secrets o configuracions d'entorns de bases de dades.

3. Sobreposicions per entorns:

ArgoCD pot utilitzar arxius `values.yaml` separats o modificadors per desplegar la mateixa aplicació en diferents entorns amb configuracions personalitzades.

Beneficis

- **Simplificació:** Redueix la redundància en la creació de Helm Charts.
- **Escalabilitat:** Fàcil adaptació a nous productes o entorns.
- **Consistència:** Garanteix una estructura uniforme per a tots els components i entorns.
- **Mantenibilitat:** Canvis centrals afecten totes les aplicacions sense modificar múltiples Helm Charts.

Falta de Definició de Resources Limits en els Deployments

Durant el procés de desplegament de les aplicacions, es detectà un biaix relacionat amb la falta de configuració explícita dels límits de recursos (resource limits) als Deployments de Kubernetes. Aquest biaix comporta riscos importants:

Impacte de la Falta de Resource Limits

En absència de límits, les aplicacions poden consumir més recursos del previst, afectant altres aplicacions que comparteixen el mateix clúster. Això pot provocar saturació de CPU o memòria, fallades en altres serveis, i fins i tot el col·lapse del node.

Sense `limits` i `requests` definits, és complicat identificar quins components estan sobreconsumint recursos. Això dificulta la resolució d'incidents en entorns de producció.

Kubernetes utilitza les mètriques de recursos per decidir quan i com escalar els pods. Si aquestes mètriques no estan ben configurades, el sistema pot no escalar correctament.

Mesures Pal·liatives

Per abordar aquest biaix, es van a implementar les següents accions:

Definir requests i limits

Es van establir valors clars per a les necessitats mínimes (`requests`) i màximes (`limits`) de cada aplicació:

```
resources:  
  requests:  
    memory: "256Mi"
```

```
cpu: "250m"  
limits:  
  memory: "512Mi"  
  cpu: "500m"
```

Això garanteix que cada aplicació tingui els recursos mínims per funcionar i eviti sobrecarregar el node.

Ús d'eines per validar configuracions:

Eines com Kube-linter[\[23\]](#) es van utilitzar per revisar els manifests de Kubernetes i assegurar-se que tots els Deployments inclouen aquests paràmetres.

1. Documentació i conscienciació:

S'ha d'actualitzar la documentació tècnica del projecte perquè aquesta pràctica sigui estàndard.

Aplicació d'AutoScaling en els Deployments

L'aplicació de AutoScaling en els Deployments és una pràctica clau en arquitectures de microserveis per ajustar automàticament els recursos segons la demanda. Això ho hem pogut aconseguir amb eines com Kubernetes Horizontal Pod Autoscaler (HPA) a les nostres templates de HELM.

Amb un YAML que configura **Horizontal Pod Autoscalers (HPA)** per als serveis Apache, MySQL i PHP d'una aplicació LAMP hem assolit el següent:

Metadades:

- Defineix un nom únic per a cada HPA basat en el `Release.Name` de Helm.
- Assigna etiquetes per identificar el component (Apache, MySQL o PHP).

Escalat Objectiu (`scaleTargetRef`):

- Indica el Deployment que l'HPA monitoritza, especificat pel `Release.Name` corresponent.

Rèpliques i Ús de CPU:

- Controla el nombre mínim i màxim de rèpliques amb `minReplicas` i `maxReplicas`.
- L'HPA escala automàticament segons el percentatge d'utilització de CPU configurat amb `targetCPUUtilizationPercentage`.

Conclusió

Tot i el biaix inicial, l'adopció d'aquestes pràctiques ha permès millorar l'eficiència del clúster i evitar possibles interrupcions. Es recomana revisar periòdicament aquests paràmetres i ajustar els límits, repliques dels pods, etc. a mesura que evolucionin les necessitats de les aplicacions.

5.2 Compliment Normatiu i Auditoria

En un entorn de microserveis i migració d'aplicacions, assegurar el compliment normatiu i la capacitat d'auditoria és essencial per mantenir la confiança dels usuaris, complir amb les lleis aplicables i protegir les dades sensibles. Aquí desenvolupem els passos clau per a implementar aquest component en el projecte:

Identificació dels Requisits Normatius

La identificació dels requisits normatius en un projecte és un procés crític per assegurar que el sistema compleix les lleis i regulacions aplicables, així com per garantir la seguretat i privadesa de les dades que es gestionen. Aquesta etapa no és simplement un tràmit legal; implica una comprensió profunda del marc normatiu i la seva aplicació pràctica dins del disseny i operació del sistema.

En el context d'aquest projecte, cal tenir en compte regulacions com el Reglament General de Protecció de Dades (RGPD)[\[24\]](#), que regeix la protecció de dades personals a la Unió Europea. Això inclou assegurar que la recopilació de dades tingui un consentiment explícit, que les dades siguin accessibles només per usuaris autoritzats i que es pugui garantir el dret a l'oblit. Per a altres regions, podrien aplicar-se normatives específiques, com la CCPA (California Consumer Privacy Act) a Califòrnia o la HIPAA (Health Insurance Portability and Accountability Act) per a dades mèdiques als Estats Units.

El procés per identificar aquests requisits comença amb una auditoria inicial per determinar quines lleis i regulacions són aplicables al projecte. Aquesta auditoria inclou l'anàlisi del tipus de dades que es processaran, ja que la naturalesa de les dades (financeres, mèdiques, personals) sol influir significativament en les obligacions normatives.

A més, la integració de normatives sectorials específiques, com els estàndards PCI DSS (Payment Card Industry Data Security Standard) [\[25\]](#) per a dades de pagament, sovint requereix consultes amb advocats i consultors especialitzats. Aquests professionals ajuden a traduir les exigències legals en requisits tècnics concrets, com ara la implementació de sistemes de xifrat o controls d'accés robustos.

La documentació d'aquests requisits és essencial per garantir una implementació coherent i revisable. Cada requisit ha de ser assignat a responsables clars dins de l'equip, assegurant que les accions necessàries es portin a terme dins dels terminis estipulats.

Un altre aspecte crític és la revisió contínua del marc normatiu. Les lleis evolucionen, i amb elles, les obligacions dels sistemes tecnològics. Per això, el sistema ha d'estar preparat per adaptar-se als canvis legislatius i incorporar les noves exigències sense alterar la seva operació principal.

Aquest procés d'identificació i implementació no només protegeix contra possibles sancions legals, sinó que també genera confiança entre els usuaris, assegurant-los que les seves dades es gestionen de manera responsable i conforme a les millors pràctiques de la indústria.

Disseny per a l'Auditoria

El disseny d'un sistema preparat per a auditories implica l'establiment de mecanismes que garanteixin la transparència, la traçabilitat i la verificació de les operacions. Això assegura que el sistema compleix amb els requisits normatius i facilita les revisions periòdiques per part d'entitats reguladores o internes.

Objectius Principals del Disseny per a l'Auditoria

1. Traçabilitat Completa de les Operacions

Cada acció significativa que succeeix en el sistema (per exemple, accessos, modificacions de dades o processos crítics) ha de ser registrada amb detalls clau, com l'usuari responsable, el temps exacte i la natura del canvi. Aquests registres han de permetre reconstruir qualsevol activitat per determinar la seva conformitat amb les polítiques establertes.

2. Seguretat dels Registres

Els registres d'auditoria han de ser protegits contra manipulacions. Això s'aconsegueix mitjançant l'ús de tècniques com el xifratge o la signatura digital, que assegurin la integritat i autenticitat de la informació.

3. Automatització de la Recollida de Dades

Per garantir l'eficàcia i evitar errors humans, els sistemes han d'automatitzar la recollida de dades d'auditoria. Aquest procés pot incloure la monitorització dels accessos al sistema, les modificacions de configuració o l'execució de transaccions.

4. Compliment Normatiu i Personalització

El sistema ha de ser capaç de recollir dades segons les especificacions de les normatives aplicables, com el RGPD o PCI DSS, ajustant les seves funcionalitats als requisits específics del sector.

Elements Clau d'un Disseny per a l'Auditoria

1. Infraestructura de Registre

Els logs han de ser centralitzats i accessibles per a anàlisi posterior. Eines com **Elasticsearch**, **Logstash** i **Kibana** (ELK Stack) poden ser utilitzades per organitzar, analitzar i visualitzar registres d'una manera eficient.

2. Integració amb Sistemes d'Alerta

Qualsevol anomalia detectada durant la monitorització ha de generar alertes automàtiques. Això pot ser configurat a través de plataformes com **Prometheus** o **Grafana**, que faciliten la detecció en temps real de problemes potencials.

3. Revisions Periòdiques i Auditories Externes

El sistema ha de ser dissenyat perquè les auditories siguin senzilles d'implementar. Això implica documentar tots els procediments, mantenir un historial clar d'actualitzacions i garantir que els registres siguin fàcilment consultables.

4. Protecció del Sistema d'Auditoria

Els logs han de ser immutables i accessibles només per personal autoritzat. Això pot incloure la configuració de rols d'usuari específics amb privilegis limitats i l'ús de serveis externs per assegurar els registres.

5. Interfície per a Auditors

Una eina específica o interfície ha de ser proporcionada als auditors per accedir de manera eficient a les dades pertinents sense exposar informació confidencial innecessària.

Exemple Pràctic

En un entorn basat en microserveis, cada servei pot disposar del seu propi sistema de registre que s'envia a un dipòsit centralitzat. Per exemple, un servei backend podria registrar totes les sol·licituds HTTP rebudes, incloent-hi dades com IP, usuari autenticat i resposta generada. Aquesta informació es pot analitzar per confirmar que s'han respectat les polítiques d'accés i s'han complert els requisits normatius.

Estratègies de Compliment Actiu

Les estratègies de compliment actiu són enfocaments proactius per garantir que un sistema o organització compleixi amb les normatives aplicables i els estàndards establerts de manera continuada i eficient. Aquestes estratègies impliquen no només la implementació de controls i mesures per satisfer els requisits normatius, sinó també la monitorització constant, l'automatització dels processos de compliment, i l'adaptació ràpida als canvis en les normatives.

Principals Estratègies de Compliment Actiu

1. Monitorització en Temps Real

Utilitzar eines avançades com **Prometheus**, **Grafana**, o sistemes d'alerta en temps real permet identificar i corregir qualsevol desviació respecte als requisits normatius de manera immediata. Per exemple, en el compliment del RGPD, es poden monitoritzar accessos no autoritzats a dades sensibles.

2. Automatització del Compliment

Mitjançant eines com **ArgoCD** o **Helm**, les configuracions normatives es poden incorporar en la infraestructura de desplegament. Això assegura que els canvis d'entorn no vulnerin cap norma i que es mantingui la conformitat durant tot el cicle de vida del sistema. Per exemple, les polítiques de seguretat poden ser desplegades i validades automàticament.

3. Controls Preventius

Implementar mesures com l'ús de frameworks de desenvolupament segur o escàners de codi com **SonarQube** ajuda a evitar vulnerabilitats abans que arribin a producció. Això també inclou verificar que el codi compleixi amb les normes aplicables abans del desplegament.

4. Formació i Sensibilització

Garantir que els desenvolupadors, operadors i personal relacionat estiguin formats en normatives com el RGPD, PCI DSS, o HIPAA assegura que les pràctiques diàries de treball estiguin alineades amb les expectatives reguladores.

5. Auditories Regulars

Planificar auditories periòdiques internes i externes ajuda a identificar àrees de millora i prepara el sistema per a revisions oficials. L'ús d'eines com **Splunk** per analitzar registres facilita aquestes auditories.

6. Adaptabilitat a Normatives Dinàmiques

Les normatives evolucionen amb el temps, per la qual cosa el sistema ha de ser flexible per incorporar canvis ràpids en requisits. Això es pot assolir amb una arquitectura basada en microserveis, on cada component es pot actualitzar de manera independent.

7. Gestió de la Traçabilitat

Mantenir un registre complet i accessible de totes les operacions, decisions, i configuracions és clau per demostrar el compliment actiu en qualsevol moment. Això inclou també la gestió d'accessos i canvis al sistema.

Exemple Pràctic

Suposem que l'organització migra una aplicació monolítica LAMP a un entorn basat en Kubernetes amb ArgoCD. Una estratègia de compliment actiu pot incloure:

- **Automatització de Configuració:** Definir plantilles d'Helm Charts que incloguin configuracions per garantir la seguretat de les dades, com polítiques de xifratge en repositori i en trànsit.
- **Monitorització Proactiva:** Configurar regles en Grafana per alertar d'activitats anòmales, com un increment sobtat d'accessos a fitxers sensibles.
- **Auditoria i Traçabilitat:** Integrar un sistema de registres centralitzat com ELK Stack per assegurar que tots els canvis són registrats amb l'autor i la justificació corresponent.

Auditories Periòdiques

Les auditories periòdiques són una part fonamental de la gestió i manteniment de sistemes tecnològics, especialment en projectes que han de complir normatives específiques i garantir la qualitat operativa. Aquestes auditories permeten assegurar que el sistema continua funcionant segons les expectatives i que s'alinea amb les polítiques

normatives i d'empresa.

Objectius de les Auditories Periòdiques

- **Garantir el compliment normatiu:** Assegurar que el sistema continua ajustant-se a normatives com el RGPD, PCI DSS, o altres requeriments sectorials.
- **Identificar problemes potencials:** Localitzar desviacions o errors abans que es converteixin en problemes més greus.
- **Validar el rendiment:** Comprovar que el sistema manté els nivells de rendiment acordats, fins i tot sota càrregues imprevistes.
- **Revisar la seguretat:** Detectar possibles vulnerabilitats o accessos no autoritzats.
- **Assegurar la traçabilitat:** Verificar que els registres (logs) contenen informació suficient per traçar canvis i activitats.

Com es Realitzen les Auditories

1. **Definició d'Objectius:** Abans de l'auditoria, es defineixen els aspectes clau a revisar, com l'ús de recursos, les configuracions de seguretat, o el compliment normatiu.
2. **Eina d'Auditoria:** S'utilitzen eines específiques com Splunk, ELK Stack o Kubernetes Audit Logs per recollir i analitzar dades.
3. **Anàlisi de Registres:** Es revisen els registres generats per les aplicacions, els sistemes operatius i els serveis relacionats. Aquesta informació és essencial per identificar anomalies o problemes.
4. **Simulació de Casos:** Es poden dur a terme simulacions d'escenaris, com ciberatacs o pics de càrrega, per validar el comportament del sistema.
5. **Generació d'Informes:** Es prepara un informe detallat amb els resultats de l'auditoria, incloent recomanacions específiques per millorar.

Avantatges de les Auditories Periòdiques

- **Millora contínua:** Detecten oportunitats per optimitzar el sistema.
- **Compliment proactiu:** Ajuda a preparar-se per auditories oficials.
- **Reducció de riscos:** Mitiguen possibles impactes negatius relacionats amb incompliments o problemes tècnics.
- **Fiabilitat:** Augmenten la confiança dels usuaris i stakeholders en el sistema.

Exemple Pràctic

En un projecte de migració d'una aplicació monolítica LAMP a un entorn basat en Kubernetes amb ArgoCD i Helm, una auditoria periòdica podria:

- Verificar que els Helm Charts compleixen amb les bones pràctiques de seguretat.
- Revisar que els recursos configurats per als microserveis són adequats per evitar sobrecàrregues o desbordaments.
- Comprovar que els permisos i l'accés als recursos estan correctament restringits.
- Assegurar que els logs estan centralitzats i accessibles per auditar qualsevol esdeveniment.

Utilització de Helm i ArgoCD per Automatitzar el Compliment

L'ús de **Helm** i **ArgoCD** en un entorn Kubernetes permet automatitzar i assegurar el compliment normatiu i tècnic de manera eficient. Aquests dos components són fonamentals per gestionar el desplegament i la configuració de serveis de manera consistent, repetible i conforme a les especificacions del projecte.

Helm: Gestió de Configuracions i Compliment Normatiu

Helm, considerat el "gestor de paquets per Kubernetes", facilita la creació, compartició i actualització de configuracions en forma de Helm Charts. Aquests Charts defineixen la infraestructura i configuracions necessàries per executar una aplicació en Kubernetes.

- **Centralització de Configuracions:** Els Helm Charts poden incloure plantilles genèriques que es poden reutilitzar en diferents aplicacions, reduint errors humans. Per exemple, un fitxer values.yaml pot definir els límits de recursos, polítiques d'accés o requisits de privacitat.
- **Automatització del Compliment Normatiu:** Configuracions predefinides per al xifrat de dades, autenticació o altres requisits normatius es poden implementar automàticament. Això és especialment útil per al compliment de normes com el RGPD.
- **Traçabilitat i Versió:** Les versions de Helm Charts permeten auditar canvis i garantir que es manté una configuració coherent en els entorns.

ArgoCD: Automatització del Desplegament

ArgoCD és una eina declarativa per desplegar aplicacions en Kubernetes seguint el model GitOps. ArgoCD fa que el compliment sigui part del cicle de vida del desplegament mitjançant integració amb Helm i gestió automàtica dels estats declaratius.

- **GitOps com a Font Única de Veritat:** Amb ArgoCD, el repositori Git actua com la font única de configuracions aprovades. Això assegura que tots els desplegaments siguin consistents i reflecteixin les especificacions normatives.
- **Monitorització i Detecció de Derives:** ArgoCD verifica que l'estat de les aplicacions en Kubernetes coincideixi amb la configuració declarada en Git. Si es detecten desviacions, les pot corregir automàticament.
- **Integració amb Helm:** ArgoCD pot gestionar Helm Charts, permetent desplegar components com bases de dades, frontends i backends seguint configuracions predefinides.

Automatització del Compliment amb un Exemple Pràctic

En un projecte de migració d'un sistema LAMP a microserveis en Kubernetes:

1. Helm Charts Reutilitzables:

- Es creen Helm Charts per als serveis bàsics (frontend, backend, base de dades), on un fitxer de configuració global defineix els entorns (producció, desenvolupament).
- Les polítiques de seguretat, com límits de recursos o encriptació, s'inclouen en plantilles Helm reutilitzables.

2. Gestió Declarativa amb ArgoCD:

- Es configura ArgoCD per sincronitzar els Helm Charts amb l'estat del clúster Kubernetes.
- ArgoCD verifica que les polítiques de compliment (com limitar l'accés a certs nodes) estiguin sempre aplicades.

3. Auditoria Automàtica:

- Les eines d'ArgoCD i Helm registren els canvis en el sistema, permetent als auditors revisar l'historial complet de configuracions i canvis aplicats.

Eines Recomanades i Beneficis Clau del Compliment Normatiu

El compliment normatiu en un sistema tecnològic modern no només requereix eines adequades, sinó també un enfocament global que cobreixi totes les fases del procés. A través de la integració d'eines específiques i estratègies clares, es pot garantir que les operacions del sistema siguin auditable i conformes amb les regulacions aplicables.

Per assegurar-se que el sistema compleix amb les regulacions aplicables, cal començar per identificar els requisits normatius. Aquesta tasca es facilita mitjançant eines de gestió documental com **Jira** o **Confluence**, que permeten mantenir un registre centralitzat i traçable dels requisits. Altres eines especialitzades com Compliance.ai poden ajudar a rastrejar regulacions específiques, com el RGPD o normes sectorials concretes.

Aquest procés aporta claredat des del principi i assegura que els equips tècnics i legals treballin conjuntament per garantir el compliment normatiu. A més, simplifica la comunicació amb auditors externs i redueix el risc d'incompliment.

El disseny del sistema ha de tenir en compte la necessitat d'una auditoria futura. Les eines com **Open Policy Agent (OPA)** s'integren al sistema per aplicar polítiques de seguretat i accés. Així mateix, Kubernetes proporciona un sistema robust de registres d'auditoria, que captura totes les activitats importants dins del clúster.

Mitjançant eines com **Auditd**, es poden mantenir registres detallats d'esdeveniments al sistema operatiu, oferint una visibilitat completa i facilitant el procés d'auditoria. Això permet que les organitzacions estiguin preparades per respondre ràpidament davant qualsevol sol·licitud de verificació externa.

Per garantir que el compliment normatiu sigui actiu i constant, les eines d'automatització juguen un paper fonamental. Helm permet definir configuracions normatives reutilitzables en les aplicacions desplegades, mentre que **ArgoCD** assegura que aquestes configuracions s'apliquin de manera consistent en cada entorn.

Aquestes eines treballen conjuntament per automatitzar el compliment normatiu, estalviant temps i reduint errors humans. Terraform, amb el seu enfocament d'infraestructura com a codi, també afegeix una capa de seguretat en incloure les normes de compliment directament en el disseny de la infraestructura.

El seguiment periòdic és essencial per identificar possibles desviacions. Aquí, l'ús de **Kubernetes Audit Logs** i altres sistemes de registre detallat permet revisar contínuament el funcionament del sistema. Les auditories regulars proporcionen una avaluació independent de l'estat del compliment, identificant àrees que necessiten millores.

Amb eines com **Auditd** i Helm, les auditories són més eficients, ja que es poden consultar registres consolidats i configuracions normatives ben documentades. Aquest enfocament no només ajuda a complir amb els requisits legals, sinó que també millora la seguretat i la fiabilitat general del sistema.

L'ús combinat de **Helm** i **ArgoCD** és clau per assegurar que les polítiques normatives es despleguin automàticament amb cada aplicació. Helm permet definir plantilles reutilitzables per a diferents entorns, assegurant que només cal modificar un fitxer de configuració global per adaptar-se a requisits específics de cada projecte o regulació.

ArgoCD gestiona aquests desplegaments en temps real, assegurant que totes les configuracions s'apliquin correctament i que qualsevol desviació sigui detectada immediatament. Aquesta integració millora enormement l'eficiència i redueix la probabilitat d'errors humans.

6. Futur del Projecte

El futur del projecte es basa en tres àmbits clau: **manteniment continuat**, **adopció de tecnologies emergents** i **optimització d'operacions**. A mesura que l'aplicació passa de la fase de migració a una operació estable, el focus es desplaçarà cap a millorar el rendiment i ampliar les funcionalitats per acomplir noves necessitats de negoci.

Evolució Tecnològica

Amb l'adopció de microserveis, el projecte està ben posicionat per integrar futures innovacions tecnològiques, com ara intel·ligència artificial per a la presa de decisions basada en dades o millores en l'observabilitat amb eines més avançades com Prometheus i Grafana. La modularitat del sistema facilita actualitzacions i canvis sense interrupcions massives.

Escalabilitat i Creixement

El sistema ja dissenyat permet escalar fàcilment per adaptar-se a augments de la demanda. Es pot ampliar la infraestructura amb Kubernetes, i les configuracions de Helm i ArgoCD proporcionen la flexibilitat necessària per desplegar nous serveis amb rapidesa i consistència.

6.1 Resum del Progrés

El projecte ha aconseguit superar els objectius inicials malgrat alguns reptes. Els punts destacats inclouen:

1. **Migració tècnica reeixida:** La transició des del monòlit a una arquitectura de microserveis ha estat implementada amb èxit.
2. **Automatització eficient:** Helm ha estat integrats per automatitzar el desplegament i garantir consistència en els entorns, malgrat que ArgoCD no ha estat possible provar-ho plenament en el laboratori per falta de recursos.
3. **Compliment normatiu assegurat:** S'han testejat algunes eines adaptades per Kubernetes, el sistema hauria de complir els mínims requerits del RGPD i altres normatives sectorials.
4. **Estratègies de prova completes:** Des de proves unitàries fins a proves de resistència i seguretat, el sistema ha estat sotmès a rigorosos controls per assegurar el seu funcionament òptim.
5. **Adopció del treball col·laboratiu:** Mitjançant eines com Jira, Confluence i CI/CD pipelines, els equips tècnics han millorat la coordinació i reduït el temps de lliurament.

6.2 Projecció de Millores Potencials i Ampliacions

En el futur, el projecte pot continuar creixent i millorant en diversos àmbits per garantir una major eficiència, flexibilitat i adaptabilitat als canvis del mercat i les noves necessitats dels usuaris. Algunes àrees de millora i possibles ampliacions inclouen:

Optimització de la Infraestructura

L'ús de Kubernetes i la infraestructura basada en microserveis ha demostrat ser una elecció robusta per a escalar l'aplicació. Tanmateix, es poden explorar noves estratègies d'optimització, com l'ús de **Vertical Pod Autoscaler (VPA)** per gestionar millor els recursos de manera dinàmica, ajustant el nombre de pods en funció de la demanda real del sistema. A més, incorporar **serverless computing** en algunes parts del sistema podria reduir costos i millorar l'escalabilitat per a tasques d'ús esporàdic.

Integració amb Intel·ligència Artificial i Machine Learning

Una de les àrees més prometedores per a una expansió futura és la incorporació de tecnologies d'**Intel·ligència Artificial i Machine Learning**. Es poden implementar models predictius per optimitzar el rendiment, millorar l'experiència de l'usuari o fins i tot per

detectar i resoldre incidències de manera automàtica abans que afectin els usuaris. Això podria incloure sistemes d'alertes intel·ligents, recomanacions personalitzades per als usuaris o una gestió més eficient de la infraestructura.

Milliores en l'Automatització amb DevOps i CI/CD

Tot i que ja s'ha implementat un sistema robust de CI/CD, s'hi podrien afegir **milliores en la seguretat i control de qualitat** a través de pipelines més avançats. Per exemple, s'integrarien **proves de seguretat automàtiques** dins del procés de desenvolupament per garantir que les vulnerabilitats es detectin més ràpidament. L'ús de **Helm charts** més modulars i configurables també pot facilitar encara més la reutilització de codi i la consistència entre diferents entorns.

Expansió del Compliment Normatiu

A mesura que el sistema evoluciona i s'expandeix a nous mercats, l'ampliació del compliment normatiu serà clau. Això inclou l'adaptació a **noves normatives internacionals** com la **Llei de Protecció de Dades Personals a Califòrnia (CCPA)** o altres regulacions locals. L'automatització d'aquests processos a través d'**auditories automatitzades** i eines com **Helm** o **ArgoCD** podrien facilitar l'adhesió continua i millorada.

Augment de la Resiliència i la Continuïtat del Servei

La **resiliència del sistema** es pot millorar a través de l'adopció de **estratègies de contingència** més robustes, com ara l'ús de **multi-cloud deployment** per evitar la dependència d'un sol proveïdor de núvol, així com implementar **estratègies de backup i recuperació ante desastres** més eficients. Això pot incloure configuracions de failover automàtic entre múltiples regions de núvol o centres de dades.

Expansió a Nous Mercats i Plataformes

El projecte pot expandir-se a noves **plataformes mòbils** o incorporar integracions amb serveis de tercers per a millorar l'experiència de l'usuari. A més, pot ser beneficiosa l'adopció de **multicanal** o **multi-llenguatge**, oferint una millor cobertura en diversos mercats geogràfics.

Ampliació de Funcionalitats per a Microserveis

Com els microserveis ja estan sent utilitzats, una ampliació futura podria consistir en la creació de nous microserveis per a funcions específiques, com ara **anàlisis de dades avançades** o gestió d'usuaris, i la seva integració amb eines de **monitorització avançada** com Prometheus, Grafana i Elastic Stack per obtenir més visibilitat i control sobre el sistema.

Millora de l'Experiència d'Usuari

Finalment, les millores de l'experiència d'usuari (UX) poden estar en el centre de les ampliacions futures. Integrar noves funcionalitats d'intel·ligència artificial o interacció amb l'usuari pot millorar la personalització del servei i augmentar la satisfacció dels usuaris. També es podrien millorar les interfícies de **front-end**, buscant una major agilitat i personalització d'acord amb les necessitats de l'usuari final.

En resum, el projecte està ben posicionat per seguir creixent i adaptant-se als canvis del mercat i les noves necessitats tecnològiques. A mesura que el sistema maduri, les millores en l'escalabilitat, l'automatització, i la integració amb noves tecnologies permetran que el projecte continuï sent competitiu i eficient a llarg termini.

7. Conclusió

En aquest treball, hem abordat el procés de migració d'una aplicació monolítica a una arquitectura de microserveis, utilitzant tecnologies com Kubernetes, Helm, ArgoCD i CI/CD, amb l'objectiu d'augmentar l'escalabilitat, flexibilitat i eficiència operativa. A través de la implementació d'aquests processos i eines, hem après diverses lliçons valuoses.

Lliçons apreses

Una de les principals lliçons apreses durant aquest projecte ha estat la importància d'una correcta planificació i disseny previ. Els primers passos del projecte, que inclouen l'anàlisi de l'arquitectura actual i la definició de l'estratègia de microserveis, són essencials per garantir una migració exitosa. També hem après que, tot i els beneficis evidents dels microserveis, la seva implementació implica un desafiament considerable, especialment pel que fa a la gestió de dependències entre serveis, la configuració de Kubernetes i el manteniment de la infraestructura.

Un altre aprenentatge important ha estat l'ús de Helm i ArgoCD per a la gestió de la infraestructura i la implementació automatitzada de les aplicacions. La reutilització de Helm charts ha permès que els components del sistema siguin més modulars i configurables, facilitant el desplegament d'aplicacions en diferents entorns amb mínima personalització.

Reflexió crítica sobre l'assoliment dels objectius

Els objectius inicials del projecte es basaven en migrar l'aplicació monolítica a una arquitectura de microserveis i automatitzar el flux de CI/CD per millorar la flexibilitat i la rendibilitat operativa. Els objectius generals s'han aconseguit en gran part. La migració s'ha completat amb èxit, tot i que s'ha requerit més temps i recursos dels previstos inicialment, especialment en l'adopció de Kubernetes, que va exigir més ajustos de configuració i gestió del clúster del que es preveia.

No obstant això, no tots els objectius inicials es van poder assolir completament. Per exemple, la implementació completa de l'escala d'alguns microserveis no es va poder realitzar durant la primera fase del projecte, degut a la complexitat addicional en la gestió de la infraestructura multi-nuol. Això es deu a la necessitat d'una planificació més detallada per escalar els serveis de forma òptima i gestionar la infraestructura multi-clúster, una tasca que requereix més temps i recursos que els previstos.

Anàlisi crítica de la planificació i metodologia

Durant el desenvolupament del projecte, la planificació inicial va ser essencial per establir les bases i garantir que les tasques es realitzessin de manera ordenada. Tot i això, es van produir desviacions en els terminis, principalment degudes a la complexitat de la configuració de Kubernetes i a la integració de les eines de CI/CD com ArgoCD. Per solucionar aquests imprevistos, vam ajustar el cronograma, afegint més temps a la fase d'implementació i concentrant-nos en les tasques més crítiques.

La metodologia utilitzada, basada en les pràctiques de DevOps i la integració contínua, va ser adequada, però va requerir certs ajustos per tal d'integrar millor les eines d'automatització en el flux de treball. Això inclou la millora de l'ús de Helm charts per aconseguir una configuració més modular i reutilitzable, així com l'automatització de les proves de seguretat i rendiment dins del procés de CI/CD.

Línies de treball futur

Algunes línies de treball futur inclouen l'optimització de l'escalabilitat i la seguretat del sistema, a mesura que creixi la base d'usuaris i el nombre de microserveis. Es pot explorar la implementació de *serverless computing* en parts de l'arquitectura per reduir costos i millorar la flexibilitat. També s'ha de treballar en la consolidació de l'integració de microserveis, amb l'objectiu de crear una arquitectura més resilient i eficient.

Altres àrees que no s'han pogut explorar completament són la millora de les estratègies de contingència per a la recuperació davant desastres i la creació de plataformes de monitorització més avançades que permetin una observabilitat 360° del sistema, incloent l'ús d'intel·ligència artificial per detectar anomalies en temps real.

En conclusió, tot i que hem aconseguit els objectius generals del projecte, les dificultats i els ajustos realitzats durant la implementació ens han proporcionat valuoses lliçons per a futurs projectes. La migració a microserveis és un procés continu, que implica una millora constant i una adaptació a les noves tecnologies i requisits del mercat.

8. Glossari

Cloud (núvol)

El terme Cloud fa referència a l'ús de serveis i recursos computacionals proporcionats a través d'Internet. Aquests serveis poden incloure emmagatzematge, bases de dades, serveis d'infraestructura i aplicacions, i poden ser utilitzats sense necessitat de mantenir la infraestructura física local. Els proveïdors de Cloud més coneguts són Amazon Web Services (AWS), Google Cloud i Microsoft Azure.

On-premises

On-premises es refereix a una infraestructura o solució tecnològica que s'instal·la i es gestiona localment dins de les instal·lacions físiques d'una empresa o organització, en lloc de ser allotjada en el núvol o en un centre de dades extern. Això implica que els servidors, bases de dades i altres recursos informàtics estan físicament presents en les instal·lacions de l'empresa i són responsables de la seva pròpia gestió, manteniment i seguretat.

Arquitectura Monolítica (Monolithic Architecture)

L'arquitectura monolítica és un enfocament tradicional en què tota una aplicació es construeix com una sola unitat de codi, on els diferents mòduls i components estan estretament interconnectats i depenen uns dels altres. Això pot fer que l'evolució i el desplegament de noves funcionalitats siguin més difícils, ja que un canvi en una part de l'aplicació pot afectar altres parts.

Arquitectura Microserveis (Microservices Architecture)

Els microserveis són una arquitectura de disseny d'aplicacions que divideix una aplicació monolítica en diversos serveis petits i independents. Cada servei es centra en una funcionalitat específica i comunica amb altres serveis a través de protocols lleugers com HTTP. Aquest enfocament facilita l'escalabilitat, la flexibilitat i la independència de les actualitzacions.

Docker

Docker és una plataforma que permet empaquetar aplicacions amb totes les seves dependències en contenidors, facilitant la seva execució de manera consistent en qualsevol entorn.

Kubernetes

Kubernetes és una plataforma d'orquestració de contenidors que permet automatitzar la implementació, l'escalabilitat i la gestió d'aplicacions contenidoritzades. Creat per Google, és un estàndard de facto per a gestionar aplicacions basades en microserveis, ajudant en la gestió de clústers de contenidors amb característiques com l'autoscaling,

la distribució de càrrega i la tolerància a fallades.

Single Responsibility Principle (SRP)

El principi de responsabilitat única (SRP) és un dels principis fonamentals de la programació orientada a objectes, que estableix que cada mòdul o classe ha de tenir una única responsabilitat, és a dir, ha de ser responsable d'una única funcionalitat. Aquest principi millora la llegibilitat i mantenibilitat del codi, ja que facilita el seu testatge i modificació. Seguir el SRP ajuda a evitar el codi inflat i difícil de gestionar.

CI/CD (Continuous Integration / Continuous Deployment)

Les pràctiques de CI i CD tenen com a objectiu millorar el procés de desenvolupament de programari. CI implica la integració de codi constantment en una branca comuna, mentre que CD s'encarrega d'automatitzar el procés de desplegament d'aquest codi a entorns de producció.

DevOps

DevOps és una metodologia que promou la col·laboració entre els equips de desenvolupament de programari i els equips d'operacions per automatitzar i millorar el procés de creació, prova i desplegament d'aplicacions. DevOps integra les pràctiques de CI/CD i utilitza eines com Kubernetes, Docker, i Helm per millorar l'eficiència en el cicle de vida del programari.

GitOps

GitOps és una pràctica de gestió d'infraestructures i aplicacions que utilitza repositoris Git com a font única de veritat per gestionar els canvis a les aplicacions i la infraestructura. Les eines com ArgoCD i Flux permeten aplicar automatitzacions de desplegament i mantenir l'estat desitjat de les aplicacions directament des de les configuracions emmagatzemades a Git.

Helm

Helm és una eina de gestió de paquets per a Kubernetes. Utilitza Helm charts, que són conjunts de fitxers que defineixen les configuracions d'aplicacions i serveis en Kubernetes. Els Helm charts permeten automatitzar la instal·lació, configuració i actualització d'aplicacions dins del clúster.

ArgoCD

ArgoCD és una eina de continuïtat d'integració i desplegament (CI/CD) basada en el model GitOps. Permet la gestió automàtica del cicle de vida de les aplicacions Kubernetes a partir de configuracions guardades en repositoris Git.

LAMP Stack

LAMP és un conjunt de tecnologies utilitzades en el desenvolupament web. Està compost per Linux, Apache, MySQL i PHP. Aquest conjunt és tradicionalment utilitzat per crear aplicacions monolítiques en entorns web.

Front-End

El front-end fa referència a la part visible i interactiva d'una aplicació o pàgina web, que és gestionada pel navegador de l'usuari. Són les tecnologies i components amb els quals l'usuari final interactua, com ara HTML, CSS, JavaScript i frameworks com React, Angular o Vue.js. El front-end gestiona la interacció amb l'usuari i comunica amb el back-end per obtenir dades o realitzar accions.

Back-End

El back-end fa referència a la part del servidor d'una aplicació que maneja la lògica de negocis, gestiona les bases de dades i interactua amb el front-end. El back-end és responsable de gestionar la comunicació entre el client i la base de dades, i de realitzar la lògica de processament de dades i operacions. Tecnologies comunes inclouen Node.js, Java, Python i Ruby.

Middleware

Middleware es refereix a un conjunt de programes que actuen com a pont entre el sistema operatiu i les aplicacions d'una xarxa. El middleware facilita la comunicació i gestió de dades entre aplicacions independents, proporcionant serveis com autenticació, seguretat, transaccions i comunicacions entre sistemes. En aplicacions web, el middleware pot gestionar peticions HTTP, bases de dades i operacions de sessions.

Base de Dades Descentralitzada (Decentralized Database)

Una base de dades descentralitzada és aquella en la qual les dades no es gestionen des d'un únic punt central, sinó que estan distribuïdes a través de múltiples nodes. Aquest model millora la resistència i la disponibilitat, ja que en cas que un node falli, altres poden continuar funcionant. Les bases de dades descentralitzades són comuns en entorns de blockchain o sistemes distribuïts.

Domain Driven Design (DDD)

Domain Driven Design (DDD) és una metodologia per a dissenyar aplicacions basades en el model del domini. DDD promou la creació de models detallats de l'àrea de negoci, que ajuden a dissenyar solucions més adequades als requisits del negoci. Fomenta la creació de subdominis i utilitza termes del negoci com a part integral del disseny del sistema.

API (Application Programming Interface)

Una API és un conjunt de regles i protocols que permeten a diferents aplicacions o serveis comunicar-se entre ells. Les APIs defineixen com una aplicació ha d'interactuar amb una altra, especificant les sol·licituds que pot fer i les respostes que pot esperar. Les APIs són crucials en els entorns de microserveis, ja que permeten la comunicació entre els

API Gateway

Un API Gateway és un component arquitectònic que gestiona totes les peticions entrants i les redirigeix als microserveis adequats. També s'encarrega de la seguretat, l'autenticació i la validació de les sol·licituds.

REST (Representational State Transfer)

REST és un estil arquitectònic per a la construcció d'API que es basa en l'ús de recursos identificats per URL i operacions estàndard com GET, POST, PUT i DELETE. REST és sense estat, el que significa que cada petició es realitza independentment, sense conservar informació de l'estat entre peticions. Això el fa lleuger i escalable. REST s'utilitza àmpliament per a les aplicacions web modernes.

gRPC (Google Remote Procedure Call)

gRPC és un sistema d'invocació remota de procediments desenvolupat per Google que utilitza HTTP/2 i Protocol Buffers (Protobuf) per a la serialització de dades. A diferència de REST, gRPC permet la comunicació entre serveis amb millor rendiment i eficiència, especialment en entorns on la latència és crítica. Proporciona una manera més compacta i ràpida de comunicar-se entre microserveis.

Flannel

Flannel és una solució de xarxa per a Kubernetes que crea una xarxa de superposició entre els contenidors desplegats en diferents nodes d'un clúster.

Monitorització i Observabilitat

La monitorització es refereix a la supervisió de la salut del sistema a través de mètriques i logs, mentre que l'observabilitat implica una visió profunda de l'estat intern del sistema per identificar i resoldre problemes

HashiCorp Vault

HashiCorp Vault és una eina de codi obert per a la gestió de secrets, com ara contrasenyes, claus d'API i certificats. Vault permet emmagatzemar, gestionar i distribuir secrets de manera segura, amb funcions de criptografia i control d'accés basat en rols. És una solució útil en entorns on la seguretat i el compliment normatiu són clau.

OAuth2

OAuth2 és un protocol d'autenticació que permet a les aplicacions accedir a recursos protegits en nom de l'usuari, sense exposar les seves credencials. OAuth2 és àmpliament utilitzat per a la integració de serveis d'autenticació com Google, Facebook i Twitter amb altres aplicacions. Aquest protocol és la base de molts sistemes d'inici de sessió únic (SSO) moderns.

Apache Kafka

Apache Kafka és una plataforma de transmissió d'esdeveniments distribuïda, de codi obert, que permet gestionar fluxos de dades en temps real. Kafka s'utilitza per a la transmissió d'esdeveniments i missatges entre microserveis, permetent la creació d'arxius de registre distribuïts i l'automatització de processos en entorns de dades massives. Kafka és molt utilitzat per l'anàlisi de dades en temps real.

9. Bibliografia

1. NEWMAN, SAM – **Monolith to Microservices** - Octubre 2019 (O'REILLY Editorial)
2. MICROSOFT- **Microservices architecture design** –
<https://learn.microsoft.com/en-us/azure/architecture/microservices>- visitat 04/11/2024
3. BLANCO CUADRADO, DANIEL SANTIAGO- **WSO2 API IS Tutorial: Cómo usar JWT Tokens para intercambiar datos**- <https://www.chakray.com/es/wso2-api-jwt-tokens-intercambiar-datos/> - visitat el 14/11/2024
4. RODRIGUEZ CALLE, NOEL- **Patrón Saga en Arquitectura de Microservicios** -
<https://refactorizando.com/patron-saga-en-arquitectura-de-microservicios> - visitat el 19/11/2024
5. seepanarajvskq - **Microservices Communication with Apache Kafka in Spring Boot**- <https://www.geeksforgeeks.org/microservices-communication-with-apache-kafka-in-spring-boot> - visitat el 21/11/2024
6. MCLUCKIE, CRAIG- **From Google to the world: The Kubernetes origin story** – Gener 2023 (O'REILLY Editorial)-
<https://cloud.google.com/blog/products/containers-kubernetes/from-google-to-the-world-the-kubernetes-origin-story>- visitat el 23/11/2024
7. VINTO, NATALE i SOTO BUENO, ALEX- **GitOps Cookbook** – Gener 2023 (O'REILLY Editorial)
8. PENCHIKALA, SRINI - **Domain Driven Design and Development In Practice**-
<https://www.infoq.com/articles/ddd-in-practice/>- visitat el 01/12/2024
9. BITLOOPS- **Domain-Driven Design**- <https://bitloops.com/docs/bitloops-language/learning/software-design/domain-driven-design>- visitat el 05/12/2024
10. FOWLER, MARTIN- **Patterns of Enterprise Application Architecture** - Març 2003
11. AMAZON WEB SERVICES - **Disaster Recovery of Workloads on AWS: Recovery in the Cloud**- <https://docs.aws.amazon.com/whitepapers/latest/disaster-recovery-workloads-on-aws/disaster-recovery-options-in-the-cloud.html>- visitat el 06/12/2024
12. DARIO, ANDERSON- **ArgoCD App of Apps – A GitOps Approach**-
<https://medium.com/@andersondario/argocd-app-of-apps-a-gitops-approach-52b17a919a66>- visitat el 08/12/2024
13. HASHICORP VAULT- <https://www.vaultproject.io/>- visitat el 11/12/2024
14. BOGAN, MICHAEL- **How to Maintain Quality when Transitioning from Monolith to Microservices** - <https://dev.to/mbogan/how-to-maintain-quality-when-transitioning-from-monolith-to-microservices-ap0>- visitat el 14/12/2024
15. FOSS TechNix (Free,Open Source Software's and Technology Nix*) - **What is HELM | Why We need HELM | Create HELM Chart?** -
<https://www.fosstechnix.com/what-is-helm-why-we-need-helm-helm-charts/>- visitat 16/12/2024
16. FRÄNKEL, NICOLAS- **Integration Testing from the Trenches**
17. FOWLER, MARTIN- <https://martinfowler.com/microservices>- visitat 19/12/2024
18. BLACK, RYAN- **soak testing**-
<https://www.techtarget.com/searchsoftwarequality/definition/Soak->

[testing#:~:text=Soak%20testing%20is%20a%20type,a%20form%20of%20stress%20testing](#) - visitat 19/12/2024

19. OWASP (Open Web Application Security Project) – **WSTG (Web Security Testing Guide)- v4.1- Penetration Testing Methodologies**- https://owasp.org/www-project-web-security-testing-guide/v41/3-The_OWASP_Testing_Framework/1-Penetration_Testing_Methodologies - visitat 20/12/2024
20. OWASP (Open Web Application Security Project)- **ZAP (Zed Attack Proxy) by checkmarx** - <https://www.zaproxy.org/> - visitat 20/12/2024
21. NIST (National Institute of Standards and Technology) - **NVD (National Vulnerability Database)** - <https://nvd.nist.gov/vuln/search> - visitat 20/12/2024
22. BEYER, BETSY , JONES, CHRIS , PETOFF, JENNIFER , MURPHY, NIAL RICHARD – **Google- Site Reliability Engineering-** (O'REILLY Editorial)- <https://github.com/euclid1990/google-sre-book/blob/master/Site%20Reliability%20Engineering.pdf>
23. **KubeLinter**- <https://docs.kubelinter.io/#/> - visitat 21/12/2024
24. **Text oficial del RGPD**- <https://eur-lex.europa.eu/legal-content/ES/TXT/?uri=CELEX%3A32016R0679> – visitat 22/12/2024
25. **Documentació de PCI DSS** - <https://www.pcisecuritystandards.org/> – visitat 22/12/2024